ML MENTOR LAB

EDUCATION WEBSITE PROJECT

Viva Question & Answer Guide

Complete page-by-page preparation, section-wise explanations, backend API references, and a dedicated walkthrough of every login method used in the Mentor Lab website.

23 page sections | Student + Admin login | OTP password recovery | React + Express + MongoDB | Mock tests, notes, certificates | AI chatbot and compiler

Prepared directly from the source code. This guide describes the actual routes, components, API endpoints, database models and validation rules found in the project. No passwords, API keys or secret values from the `.env` file are included.

Mentor Lab
BCA, BBA and Biotechnology learning platform

October 2026
Viva preparation document

CONTENTS

What This Guide Covers

PART 01

Project at a Glance

- Purpose, users and courses
- Technology stack and architecture
- Folder structure and request flow
- Environment variables and run steps
- Complete route map

PART 02

All Logins & Authentication

- Student login and signup
- Admin portal login
- Forgot password and email OTP
- Google / Firebase login status
- Route guards, JWT and logout

PART 03

Page-by-Page Q&A

- Home, Navbar, About, Contact
- Study Notes, BCA, BBA, Biotech
- Category pages and roadmaps
- Note Viewer, PYQ, Lectures
- Mock Test, Review, Analytics
- Profile, Certificates, Chatbot
- Code Compiler, Admin Dashboard

PART 04

Backend & Database Reference

- API endpoint tables
- Mongoose models and fields
- Middleware and helpers

PART 05

Feature Deep Dives

- PDF parsing and GridFS
- Payments and test unlocking
- Certificate generation logic
- Groq AI, TTS and compiler

PART 06

Security, Limits & Improvements

- Known security observations
- Scalability and code-quality notes
- Recommended next steps

PART 07

Rapid-Fire & Demo

- One-line answers
- Likely external examiner questions
- Suggested live demo order

How to use this document: Part 1 gives the big picture. Part 2 is the most important section for login-related questions. Part 3 follows the website menu order, so you can revise one page at a time. Parts 4 and 5 help with technical "how does it work internally" questions. Parts 6 and 7 prepare you for critical questions and the live demonstration.

PART 01

01

Project at a Glance & Architecture

Purpose, technology choices, folder layout, request lifecycle and the complete route map.

1.1 Project Overview

`Project name: Mentor Lab`

Q1 What is Mentor Lab?

Mentor Lab is a full-stack education website for college students, mainly covering BCA, BBA and Biotechnology. It brings syllabus material, study notes, previous-year question papers, recorded lectures, mock tests, performance analytics, certificates, an AI study assistant and a browser code compiler into one platform.

Q2 What is the main objective of the project?

The objective is to give students a single place to learn, practise and track progress. Instead of searching for notes, PYQs and mock tests separately, a student can study a subject, take a test, review mistakes, see analytics and earn a course-completion certificate. Administrators can upload content and manage paid mock-test access from one dashboard.

Q3 Who are the users of the system?

There are three practical roles: **guest** (can browse home, syllabus, notes, PYQ, chatbot and roadmaps), **student** (must log in for mock tests, analytics, profile and certificates), and **admin** (separate admin login, dashboard, content uploads and purchase management). Admin is a flag on the same User model, not a separate collection.

Q4 Which courses are supported?

Three course families: **BCA** (Programming Languages, Logical Subjects, Mathematics Subjects), **BBA** (six management/finance/marketing categories) and **Biotechnology** (Foundation Sciences, Core Biotechnology, Advanced & Applied). Each course has a syllabus page, category pages and a roadmap page.

Q5 What is the frontend technology stack?

React 19 with Vite 8, React Router DOM v7, Axios for HTTP requests, Recharts for dashboard charts, qrcode.react for the registration QR code, Firebase SDK for an unused Google-auth scaffold, and plain CSS files stored under `frontend/src/css_files`. The Vite React Compiler plugin is also enabled.

Q6 What is the backend technology stack?

Node.js with Express 5 in ES-module mode, Mongoose 9 for MongoDB, JSON Web Tokens for authentication, bcryptjs for password hashing, Multer for uploads, pdf-parse for text extraction, Nodemailer for reset-code email, Razorpay for optional online payments, and native `fetch` calls to the Groq chat-completions API.

Q7 Which database is used and why?

MongoDB is used through Mongoose. Its document model suits course content, question arrays, activity lists and flexible note metadata. Large PDF files are not stored inside normal documents; they are stored in MongoDB GridFS buckets named `pdfs` and `pyqs`.

Q8 Is this a MERN stack project?

Yes, in practice it is a MERN-style project: MongoDB, Express, React and Node.js. It is not limited to the classic MERN libraries because it also uses Vite, React Router, GridFS, Razorpay and an external AI API.

Q9 Why was React chosen for the frontend?

React allows the large number of pages to be built as reusable components, keeps the interface reactive without page reloads, and works naturally with React Router for a single-page application. It also has a large ecosystem for charts, QR codes and HTTP requests.

Q10 Why was Vite used instead of Create React App?

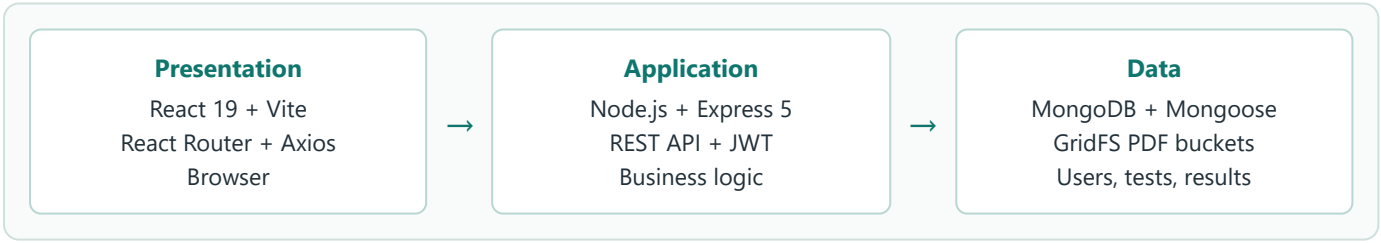
Vite gives much faster development startup and hot-module replacement, a smaller and simpler configuration, and an optimised production build. The project's `vite.config.js` also defines a development proxy for `/api` and `/uploads`, which avoids CORS issues during local development.

Q11 Why is Express used on the server?

Express provides a small, well-understood routing layer. Each feature has its own router file (`authRoutes`, `notesRoutes`, `mockTestRoutes`, and so on), which keeps the backend modular. Express also serves the built React files and uploaded PDFs from the same server.

Q12 Explain the overall architecture.

It is a three-tier architecture. The browser runs the React SPA; it calls REST endpoints on the Express server; the server validates the request, applies JWT authentication where required, and reads or writes MongoDB. External services are called only from the backend or, for the compiler, directly from the browser.



Q13 What is the folder structure of the project?

The main folder is `Monitor Lab`. It contains `backend` and `frontend`. The backend has `src` (entry and database), `routes`, `controllers`, `model`, `middleware`, `utils` and `uploads`. The frontend has `src/pages`, `src/components`, `src/data`, `src/css_files`, `src/context`, `src/firebase` and `src/utils`.

```
Monitor Lab/
  backend/
    src/index.js          Express entry + route mounting
    src/db.js             MongoDB connection
    routes/               auth, admin, mocktest, notes, pyq,
                          ai, payment, certificate
    controllers/          authController, aiController, ttsController
    model/                User, MockTest, Result, Purchase, Notes,
                          PYQ, Certificate, Setting, Course, Lecture
    middleware/           protect (JWT verification)
```

```
    utils/                generateToken, mailer
    uploads/              temporary PDFs and payment QR image
  frontend/
    src/App.jsx           Router and route list
    src/pages/            one component per page
    src/components/       Chatbot, CodeCompiler, route guards
    src/data/             BCA / BBA / Biotech category data
    src/css_files/        page-wise CSS
    src/context/          dark / light theme
    src/firebase/         optional Google-auth scaffold
```

Q14 Explain the lifecycle of a typical API request.

A React page calls Axios. In development, Vite proxies `/api` to `http://localhost:5000`. Express matches the router, runs middleware such as `protect`, executes the route handler, queries MongoDB through Mongoose, and returns JSON. The React component stores the response in state and re-renders. In production, Express serves the built `frontend/dist` files and the same API.

Q15 How does the development proxy work?

`vite.config.js` maps `/api` and `/uploads` to `http://localhost:5000`. The browser therefore sends requests to the Vite dev server, which forwards them to Express. This makes frontend code use relative paths such as `/api/auth/login` and avoids hard-coded backend URLs for most requests.

Q16 How is the website served in production?

After `npm run build`, Express serves `frontend/dist` as static files. A catch-all route `app.get("/{*path}")` sends `index.html` for any non-API path, so refreshing a client-side route such as `/mock-test` still loads the React app instead of returning a 404.

Q17 What is a single-page application?

An SPA loads one HTML document and lets JavaScript change the view. React Router swaps page components without a full page reload. The server only needs to return `index.html` for frontend routes; the API returns JSON separately.

Q18 Which environment variables does the backend use?

The main variables are `PORT`, `MONGO_URI`, `JWT_SECRET` and `GROQ_API_KEY`. Optional integrations use `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, and SMTP settings such as `SMTP_HOST`, `SMTP_USER` and `SMTP_PASS`. Secrets are kept in `.env` and loaded with `dotenv`; they should never be committed or printed in a report.

Q19 How do you run the project locally?

Start MongoDB, create `backend/.env`, then run `npm install` and `npm run dev` (or `node src/index.js`) inside `backend`. In a second terminal, run `npm install` and `npm run dev` inside `frontend`. Vite usually opens on port 5173 and proxies API calls to port 5000.

Q20 Which third-party services are integrated?

Groq for AI chat and note explanation, Google Translate TTS for text-to-speech, Razorpay for optional online payments, Nodemailer SMTP for reset codes, MongoDB Atlas or a local MongoDB server for data, and Wandbox for online code compilation. Firebase is installed and configured with placeholders but is not connected to any login button.

`Q21` Why are PDFs stored in GridFS instead of a normal folder?

Study notes and PYQ papers can be large and numerous. GridFS splits a file into chunks inside MongoDB, so it can handle files larger than the 16 MB BSON limit and keeps file metadata together with the database. It also allows streaming the PDF back to the browser without depending on a local filesystem path.

`Q22` How does dark mode work?

`ThemeProvider` stores a boolean `darkMode` (default `true`) in React context. `App.jsx` applies the class `dark-theme` or `light-theme` to the root div, and the CSS variables change accordingly. The Navbar's theme button toggles the context value.

`Q23` What are the most reusable frontend components?

The Navbar, the floating Chatbot, `CodeCompiler`, `ProtectedRoute`, `AdminRoute`, and the three course-category page components. The category pages share the same pattern but are separate files because each course has its own data source and styling.

`Q24` What is the complete route map of the frontend?

The public routes are home, BCA, BBA, biotech syllabus, category pages, study notes, PYQ, recorded lectures, about, contact, roadmaps, chatbot, login, signup, forgot password and note viewer. The protected routes are mock test, test review, analytics, profile (page-level guard via API), certificates and certificate view. The admin route is guarded by `AdminRoute`.

Route	Page	Access
`/`	Home	Public
`/bca`, `/bca/:categorySlug`	BCA curriculum and category	Public
`/bba`, `/bba/:categorySlug`	BBA curriculum and category	Public
`/biotechsylla`, `/biotechsylla/:categorySlug`	Biotech syllabus and category	Public
`/study-notes`, `/notes/:id`	Notes hub and note viewer	Public
`/pyq`, `/recorded-lectures`	PYQ portal and lectures	Public
`/about`, `/contact-us`	About and contact	Public
`/roadmapbca`, `/bbaroadmap`, `/biotech`	Three roadmaps	Public
`/login`, `/signup`, `/forgot-password`	Student auth pages	Public
`/admin-login`	Admin login	Public
`/mock-test`	Mock test portal	Logged-in student
`/test-review/:resultId`	Answer review	Logged-in student
`/analytics`	Performance analytics	Logged-in student
`/profile`	Student profile	Logged-in student
`/certificates`, `/certificate/:id`	Certificates	Logged-in student
`/chatbot`	Full-page AI assistant	Public
`/admin`	Admin dashboard	Admin only (client guard)

Viva tip: If the examiner asks "what is your project in one line", say: "Mentor Lab is a MERN-based learning platform where BCA, BBA and Biotech students can study notes, attempt mock tests, track performance and earn certificates, while admins manage content and payments."

PART 02

02

All Logins & Authentication

Every way a user can enter, register, recover an account or be blocked from a page.

2.1 The Complete Login Inventory

`Active logins: student + admin | Recovery: email OTP | Scaffolded: Google`

Q25 How many login methods does the project actually use?

There are **two active login portals**: the student login at `/login` and the admin login at `/admin-login`. Both call the same backend endpoint, `POST /api/auth/login`; the admin portal additionally checks the `isAdmin` flag. There is also an **email OTP password-recovery flow**, a **signup flow**, a **QR shortcut to signup**, and an **unused Firebase Google-auth scaffold**.

Access method	Entry point	Credential / action	Current status
Student login	`/login`	Email + password	Fully active; returns JWT
Admin login	`/admin-login`	Email + password + admin flag	Fully active; same API, extra role check
Student signup	`/signup`	Name, email, phone, password	Active; creates account and JWT
QR registration	Home page QR	Opens `/signup?ref=qr`	Active marketing shortcut, not a separate login
Forgot password	`/forgot-password`	Email OTP then new password	Active; SMTP optional, dev OTP fallback
Google login	`firebase.js`	Google provider	Scaffolded only; placeholder config, no UI button wired
Logout	Navbar	Clears local storage	Active

Q26 Is admin a separate login system?

No. Admin uses the same User collection and the same login endpoint. The difference is the boolean `isAdmin` field on the user document. The admin login page rejects a valid student account with the message "Access denied. You do not have administrative privileges."

Q27 Which routes are protected on the frontend?

`ProtectedRoute` wraps mock test, test review, analytics, certificates and certificate view. It only checks whether a token exists in `localStorage`. `AdminRoute` wraps the admin dashboard and checks both `token` and `isAdmin === "true"`. Profile and Note Viewer are not wrapped, but they call protected APIs or behave differently without a token.

Q28 What is authentication versus authorization in this project?

Authentication proves identity: login checks email and password and issues a JWT. **Authorization** decides what the identity may do: `protect` verifies the token, `isAdmin` checks decide admin actions, and `hasAccess` decides whether a student has paid for a mock test.

Q29 Which backend middleware performs authentication?

`middleware/authMiddleware.js` exports `protect`. It reads the `Authorization` header, expects the format `Bearer <token>`, verifies the token with `JWT_SECRET`, loads the user by the token's `id` without the password, and attaches it as `req.user`.

Q30 What happens if the token is missing or invalid?

If no token is present, the API returns `401` with "Not authorized, no token". If verification fails, it returns `401` with "Not authorized, token failed". The frontend route guard only checks token presence, so an expired token is usually discovered when the first protected API call fails.

Q31 Which endpoints are public and which require a token?

Public: signup, login, forgot/verify/reset password, notes listing and note detail, note PDF, PYQ listing and PDF, AI chat/explain/TTS, and payment settings. Protected: profile, add activity, mock tests and results, purchases, note read tracking, certificate generation and listing, and admin purchases. Some admin statistics and student-list endpoints are currently public, which is a known improvement area.

2.2 Student Login

Q32 Explain the student login flow step by step.

The student opens `/login`, enters email and password, and submits the form. The page sends `POST /api/auth/login` with Axios. The controller finds the user by email, explicitly selects the hidden password field, compares the password using `bcrypt.compare`, generates a JWT, and returns the user's id, name, email, `isAdmin` and token. The frontend stores the token and user in `localStorage` and navigates to the home page or the intended page.

Q33 Which API endpoint handles student login and what does it return?

`POST /api/auth/login`. On success it returns `{ _id, name, email, isAdmin, token }`. On failure it returns `400` with "Invalid email or password". The same message is used for an unknown email and a wrong password to avoid revealing which one is wrong.

Q34 How is the password verified?

Passwords are never stored in plain text. Signup hashes the password with bcrypt using 10 salt rounds. Login fetches the hash with `.select("+password")` because the schema marks password as `select: false`, then calls `bcrypt.compare(enteredPassword, storedHash)`.

Q35 Why use bcrypt instead of SHA-256 for passwords?

SHA-256 is fast and designed for integrity, which makes brute-force attacks cheaper. bcrypt is intentionally slow, uses a per-password salt, and has a configurable cost factor. The project uses 10 rounds, so each guess is computationally expensive for an attacker.

Q36 What is inside the JWT?

The token is signed with `JWT_SECRET` and has the payload `{ id: user._id }` with an expiry of **7 days**. It does not contain the password or the admin flag. The server reloads the user from the database on every protected request, so a deleted user cannot keep using an old token.

Q37 Where is the JWT stored and how is it sent?

It is stored in browser `localStorage` under the key `token`, along with a JSON copy of the user and, for admins, `isAdmin = "true"`. Protected requests send it as `Authorization: Bearer <token>`. Note that `localStorage` is readable by JavaScript, so a strong Content Security Policy and XSS protection are important.

Q38 What does the login page do with the returned user?

It stores `token` and `user` in local storage, updates the UI state, and navigates to the home page. The Navbar then detects the token and shows the logged-in menu instead of Login/Signup. If the user came from a protected page, the app may redirect back to that page.

Q39 Does the login page validate the email format and password length?

The form uses an `input type="email"` and HTML `required` attributes, and the submit handler prevents empty submissions. The backend performs the authoritative credential check. The strongest password policy is applied during password reset, not during signup.

Q40 What is the difference between the student and admin login UI?

Both use the same login CSS, but the admin page has a red heading, the text "Admin Portal Login", a red secure-login button, and an extra client-side check on `response.data.isAdmin`. A student account that logs in through the admin page receives an access-denied message and its token is not saved by that page.

Q41 Where is the admin account created?

There is a backend script, `createAdmin.js`, which creates an admin user with `isAdmin: true` and a bcrypt-hashed password. The credentials are hard-coded in that script for initial setup. For a real deployment they should be moved to environment variables, changed immediately, and the script should not be kept with a real password.

Q42 How is a user recognised as an admin after login?

The login response includes `isAdmin` from the database. The admin page stores `isAdmin = "true"` in local storage, and `AdminRoute` checks that value. On the server, admin actions such as viewing purchases or manually unlocking a test check `req.user.isAdmin` after `protect` loads the user.

Q43 Can a student simply type `/admin` in the URL?

The client-side `AdminRoute` will redirect them to `/admin-login` if the local `isAdmin` value is not "true". However, client-side checks can be bypassed, so every sensitive admin API must also verify the role on the server. Some admin routes do this; a few read-only admin routes currently do not, which should be fixed.

Q44 Why is the admin login page public?

An unauthenticated person must be able to reach the login form. The page itself is harmless; it only becomes dangerous if the credentials are weak or if admin APIs trust the client. The page does not expose a list of admins or any protected data.

2.3 Signup and Account Creation

`Q45` Explain the signup flow.

The student opens `/signup`, enters first name, optional middle name, last name, email, country code and phone number, password and confirm password. The frontend validates the fields and password strength. It sends `POST /api/auth/signup`. The backend validates names, phone length and email domain, checks for a duplicate email, hashes the password, creates the user with the student role, and returns a JWT so the student is effectively logged in.

`Q46` Which fields are stored in the User document during signup?

`firstName`, `middleName` (default empty), `lastName`, a combined `name`, `email`, `phone` (country code plus number), and the bcrypt `password` hash. `isAdmin` defaults to false. Other fields such as `college`, `studentId`, `readNotes`, `certificates` and `performanceStats` are filled later or have defaults.

`Q47` What validations does the backend apply to names?

First and last names are required and must match `/^[a-zA-Z]+(?:\s[a-zA-Z]+)*$/`, meaning letters and single spaces only. The middle name is optional but must follow the same rule if provided. This blocks numbers and special characters in names.

`Q48` How is the phone number validated?

The frontend supplies a country code such as `+91`. The backend splits the code from the number, looks up the expected digit count in a country map, and checks that the remaining part contains only digits and has exactly that length. For example, `+91` expects 10 digits. If the country is unknown, the length check is relaxed but the digits-only check remains.

`Q49` Why does the backend restrict email domains?

It maintains an allow-list of well-known providers such as Gmail, Yahoo, Outlook, Hotmail, iCloud, ProtonMail and Zoho. This catches typos such as "gmial.com", blocks disposable or fake domains, and keeps student contact addresses usable for reset codes. The trade-off is that a student with a legitimate college domain cannot register unless that domain is added to the list.

`Q50` What are the password rules?

The signup UI enforces at least 8 characters with a letter, a number and a special character, and requires the confirm password to match. The reset-password controller enforces the same strong-password rule on the server. A viva-worthy observation is that the signup endpoint itself does not currently call the strong-password function, so the server should be hardened to match the UI.

`Q51` What happens if the email is already registered?

The controller queries `User.findOne({ email })` and returns `400` with "User already exists". The email field also has a unique index in the schema, which provides a second line of defence against race conditions.

`Q52` What does signup return to the frontend?

On success it returns `201` with the new user's id, name, first name, last name, email, phone and a JWT. It does not explicitly return `isAdmin`, because a normal signup is never an admin. The frontend can store the token and treat the student as logged in.

Q53 Is email verification required before login?

No. Signup creates an active account immediately and issues a token. There is no verification link or OTP step at registration. The email is only used later for password recovery. Adding email verification would improve trust and prevent fake registrations.

Q54 How does the QR code on the home page relate to signup?

The home page uses `qrcode.react` to generate a QR code for `${window.location.origin}/signup?ref=qr`. Scanning it opens the normal signup page; the `ref=qr` query string can be used for attribution. It is a marketing shortcut, not a different authentication method.

2.4 Forgot Password and Email OTP

Q55 Describe the forgot-password flow.

The page has four steps: enter email, enter the 6-digit OTP, choose a new password, and see the success state. The backend exposes three endpoints: `POST /api/auth/forgot-password`, `POST /api/auth/verify-reset-otp` and `POST /api/auth/reset-password`. The OTP is emailed if SMTP is configured; in development, if mail delivery fails, the OTP is returned as `devOtp` for testing.

Q56 How is the OTP generated and stored?

The server uses `crypto.randomInt(100000, 1000000)` to generate a six-digit code. It stores only a bcrypt hash in `resetOtpHash`, along with `resetOtpExpires` (10 minutes), `resetOtpRequestedAt` and an attempt counter. The plain OTP is never saved in the database.

Q57 What protections exist around the OTP?

There is a 60-second resend cooldown, a 10-minute expiry, and a maximum of 5 incorrect attempts. After 5 failures or expiry, the OTP fields are cleared and the student must request a new code. These measures reduce brute-force and spam abuse.

Q58 What happens after the OTP is verified?

The server clears the OTP fields, creates a random `jti`, stores its SHA-256 hash in `resetTokenHash`, and returns a JWT signed with `JWT_SECRET` that expires in **10 minutes**. The token payload contains the user id, `purpose: "password-reset"` and the `jti`.

Q59 Why is a reset token used instead of letting the OTP reset the password directly?

It separates proof of email ownership from the password change, limits the window for the final step, and makes the token one-time. The server checks that the token purpose is correct and that the stored hash of the `jti` still matches; after a successful reset, the hash is cleared, so replaying the same token fails.

Q60 What password rules apply during reset?

`isStrongPassword` requires at least 8 characters, at least one letter, at least one digit and at least one non-alphanumeric character. The new password is hashed with bcrypt before saving, and all reset-related fields are cleared.

Q61 How does the system avoid revealing whether an email is registered?

`forgotPassword` always returns the same generic message: "If an account exists for that email, a verification code has been sent." If the email is unknown, it still returns `200` with that message. This prevents account enumeration.

Q62 What happens if SMTP is not configured?

The mail helper reports that delivery failed. The server logs a warning and, when `NODE_ENV` is not production, includes the OTP in the JSON response as `devOtp` so the flow can still be demonstrated. In production, no OTP is exposed to the client.

Q63 Can the reset flow be used to log in directly?

No. The reset token has the purpose `password-reset` and is only accepted by the reset-password endpoint. It is not stored in local storage as a normal session and cannot be used as a Bearer token to call protected APIs.

2.5 Google, Firebase and Other Access Paths

Q64 Is Google login working in the project?

No. The project contains `src/firebase/firebase.js`, which initialises Firebase and exports `auth` and `googleProvider`, but the configuration uses placeholder values such as `YOUR_API_KEY`. No page imports or renders a "Sign in with Google" button, so the flow is scaffolded but not active.

Q65 What would be needed to activate Google login?

Create a real Firebase project, replace the placeholder configuration, enable the Google provider in the Firebase console, import `signInWithPopup` and the provider, add a button, handle the returned Google user, and then either create or link a MongoDB user and issue the application's own JWT. The backend would also need an endpoint to verify the Firebase ID token if server-side trust is required.

Q66 Does the QR code count as a login method?

No. It only opens the signup page with a referral query parameter. The student still creates an account and then logs in with email and password. It is an acquisition feature, not an authentication factor.

Q67 Is there any role other than student and admin?

No. The User schema has one boolean, `isAdmin`. There is no teacher, moderator or content-editor role. Any future staff role would require either more flags or a proper role field plus server-side checks.

Q68 Can a user change their role from the profile page?

No. `updateProfile` only allows `name`, `studentId` and `college`. It does not accept `isAdmin`, password, email or statistics, so a student cannot promote themselves through the profile API.

Q69 What happens on logout?

The Navbar removes `token`, `user` and `isAdmin` from local storage and redirects to the home page. Because the token is stored client-side, logout is immediate on that browser, but the old JWT technically remains valid until its 7-day expiry if it was copied elsewhere. A production system would use short-lived access tokens plus refresh-token revocation.

Q70 Does the backend support logout or token revocation?

No. JWTs are stateless and there is no blacklist or session table. Logout only clears the browser. This is acceptable for a college project but is a limitation for a real deployment.

`Q71` What is the biggest security weakness in the current authentication design?

Storing the JWT in `localStorage` makes it accessible to any injected script, so XSS becomes token theft. There is also no refresh-token rotation, no email verification, and a few admin read endpoints are not role-protected. The strongest practical improvements are an HTTP-only secure cookie, CSRF protection, rate limiting, email verification and consistent server-side `isAdmin` middleware.

`Q72` How would you demonstrate all login paths in a viva?

First show guest browsing. Then create a student account, log out, log in, open a protected page and show the token in DevTools. Next open the admin login, show that a student account is rejected, then log in with the seeded admin and show the dashboard. Finally run the forgot-password flow and use the development OTP (or the email) to reset the password and log in again.

Most likely login question: "How does your login work end to end?" Answer in five steps: form submit to `/api/auth/login`, Mongoose lookup with password selected, bcrypt compare, JWT signed for 7 days, token stored in local storage and sent as a Bearer header to protected APIs.

PART 03

03
Page-by-Page Viva Q&A

Follows the website order. Each page has its route, purpose, main sections and likely questions.

PAGE 3.1

Home Page

`/ | Public | No API calls`

The landing page is the marketing and entry point of the platform. It introduces Mentor Lab, highlights the four main learning tools, lists the three courses and provides a scannable registration QR code.

Sections on the page

Section	What it contains
Hero	Background video `/frontvideo.mp4`, headline "A complete guide for Students", tagline, "Start Practicing" and "Browse Courses" buttons
Why Choose Us	Four clickable cards: Mock Test, Recorded Lectures, Study Notes, Performance Analytics
Popular Courses	Three cards for BCA, BBA and Biotechnology with static mock-test counts and roadmap links
Scan to Register	A generated QR code for the signup page, plus a PNG download button
Footer	Copyright line

Q73 What is the purpose of the home page?

It introduces the platform, directs visitors to the main features and courses, and converts a visitor into a registered student through the QR code and signup call-to-action. It is intentionally public so that a new visitor can explore before creating an account.

Q74 What happens when the user clicks "Start Practicing"?

It links to `/mock-test`. Because that route is wrapped in `ProtectedRoute`, a guest is redirected to `/login`, while a logged-in student goes directly to the mock-test portal.

Q75 How is the hero video implemented?

A native HTML5 `<video>` element uses `autoPlay`, `loop`, `muted` and `playsInline` with the source `/frontvideo.mp4` from the public folder. Muting is required by most browsers for autoplay; `playsInline` prevents full-screen playback on mobile.

`Q76` What do the four "Why Choose Us" cards do?

They act as shortcuts: Mock Test opens the test portal, Recorded Lectures opens the lecture page, Study Notes opens the notes hub, and Performance Analytics opens the analytics page. Each card uses CSS custom properties to give itself a different accent colour and glow.

`Q77` Are the course numbers on the home page dynamic?

No. Values such as "42 Mock Tests", "34 Mock Tests", "28 Mock Tests" and "6 Months Access" are static marketing text. They are not fetched from the database. A realistic improvement would be to count published tests per course from the backend.

`Q78` How is the registration QR code generated?

The page imports `QRCodeCanvas` from `qrcode.react` and passes `value={`${window.location.origin}/signup?ref=qr`}`. It uses error-correction level M. The canvas is stored in a React ref so the download button can call `toDataURL("image/png")` and save the image as `mentor-lab-register-qr.png`.

`Q79` Why is `window.location.origin` used instead of a hard-coded URL?

It makes the QR code point to whichever host is serving the site: localhost during development, a staging domain or the real production domain. This avoids the common mistake of printing a QR code that only works on the developer's machine.

`Q80` What is the purpose of the `#qr-register` hash handling?

If the user arrives at `/#qr-register`, the `useEffect` smooth-scrolls to the QR section. This supports a direct "register" link from another page or poster without needing a separate route.

`Q81` What could be improved on the home page?

Replace static statistics with live counts, add descriptive text to the Why Choose Us icon cards for accessibility and SEO, compress the hero video for mobile data, use local or optimised course images instead of remote Unsplash URLs, and add a visible "Login" call-to-action for returning students.

PAGE 3.2

Navbar / Global Header

`Shown on every route | Reads localStorage`

The Navbar is not a route; it is a global component rendered above every page inside the Router. It handles navigation, the course menu, theme switching, authentication state and logout.

Sections on the page

- **Brand:** Mentor Lab logo and name, linking to the home page.
- **Main links:** Home, About, Study Notes, Mock Test, PYQ and other course links.
- **Course dropdown:** BCA, BBA and Biotechnology entries with syllabus and roadmap links.
- **Auth area:** Login and Signup for guests; Profile, Dashboard (admin), Analytics and Logout for signed-in users.
- **Theme control:** toggles the dark/light context.
- **Mobile menu:** hamburger button that opens the collapsed navigation.

`Q82` How does the Navbar know whether a user is logged in?

It reads `localStorage.getItem("token")` and `localStorage.getItem("isAdmin")` when it renders. If a token exists, it shows the signed-in menu; if `isAdmin === "true"`, it also shows the admin dashboard link. It does not call a profile API just to render the header.

`Q83` How does logout work?

The logout handler removes `token`, `user` and `isAdmin` from local storage. It then redirects to `/` using `window.location.href`, which forces a full reload so the Navbar and all route guards immediately see the logged-out state.

`Q84` Why does the Navbar read local storage instead of React context?

It is a simple approach for a small project, but it means the component only updates when it re-renders, for example after navigation or a reload. A dedicated `AuthContext` would centralise the user state and update the header instantly after login or logout.

`Q85` How is the theme toggle connected to the rest of the app?

The Navbar consumes `ThemeContext`, calls its toggle function, and the provider updates `darkMode`. `App.jsx` applies `dark-theme` or `light-theme` to the root wrapper, and CSS variables change colours across all pages.

`Q86` Is the selected theme remembered after a refresh?

No. `ThemeProvider` initialises `darkMode` to `true` and does not read or write local storage. Adding persistence would require reading a saved value on initial load and saving it inside the toggle function.

`Q87` How does the course dropdown work?

It is a CSS/React-driven dropdown that reveals links to the BCA, BBA and Biotechnology pages. On mobile it becomes part of the collapsible menu. The links use React Router so navigation does not reload the page.

`Q88` What is the difference between the Navbar links for guests and students?

Guests see Login and Signup. Signed-in students see Profile, Mock Test, Analytics and Logout. An admin additionally sees the Admin Dashboard link. The change is purely based on local storage flags; the server still enforces access on protected APIs.

> `Q89` **What accessibility improvements would you suggest for the Navbar?**
>
> Add `aria-expanded` and `aria-controls` to the mobile toggle, ensure the dropdown can be operated with a keyboard, add a visible focus style, give icon-only buttons an accessible label, and make sure the contrast ratio is sufficient in both themes.

PAGE 3.3

About Us Page

`/about | Public | Static content`

The About page explains the platform's mission, offerings, values and team. The content is defined in JavaScript arrays inside the component, so it is easy to update without touching the JSX layout.

Sections on the page

Section	What it contains
Hero	Badge, headline about learning and growth, short introduction, four statistics
Mission / Offer / Why	Three cards describing the mission, the four offerings and the platform's approach
Our Values	Accessible Education, Expert-Driven Content, Continuous Innovation
Our Team	Six members with photo, name and role
CTA	"Ready to Start Learning?" with links to home and study notes

Q90 What is the purpose of the About Us page?

It builds trust by explaining who is behind Mentor Lab, what the platform offers and what values guide it. It also gives credit to the six team members and provides a final call-to-action for new learners.

Q91 Is the team information fetched from the backend?

No. It comes from a constant `team` array in `AboutUs.jsx`, with fields `name`, `role` and `photo`. Photos are served from the frontend public `team` folder. Adding a team CMS would make it dynamic.

Q92 How are the repeated cards rendered?

The component uses `stats.map`, `values.map` and `team.map`. Each item has a stable key such as `label`, `title` or `name`, which helps React update the list efficiently.

Q93 What does the "100% Free Learning Resources" stat mean?

It is a marketing claim shown in the static stats array. In the actual system, notes, PYQs and lectures are free, while some mock tests can be paid. Therefore the statement should be worded carefully or changed to "Free core learning resources" to stay accurate.

Q94 Which CSS file styles this page?

`src/css_files/about.css`. The page uses its own classes such as `about-hero`, `about-card`, `value-card`, `team-grid` and `about-cta`, in addition to the global theme variables.

Q95 What would you improve on the About page?

Replace the emoji icons with accessible SVG icons, add real social or contact links for the team, optimise and standardise team photo sizes, make the stats dynamic if possible, and avoid claiming "100%" free if paid tests exist.

PAGE 3.4

Contact Us Page

`/contact-us | Public | Client-side form only`

The Contact page provides a simple four-field form and alternative contact details. The form is a controlled React form, but it currently does not send the message to a backend or email service.

Sections on the page

- **Hero:** "Contact Us" heading and a short invitation.
- **Form:** Name, email, subject and message, with a Send Message button.
- **Other ways to reach us:** sample email, phone and address.
- **Footer:** copyright.

Q96 What happens when the contact form is submitted?

The `handleSubmit` function prevents the default page reload, shows a browser `alert` thanking the user, and resets the four fields. No HTTP request is made, so the message is not stored or emailed.

Q97 Why is this a limitation?

A real contact form should deliver the message. As written, the user believes the message was sent, but it is lost when the page closes. The project should add a backend endpoint that validates the input and sends mail through Nodemailer, or use a service such as EmailJS/Formspree.

Q98 How is the form state managed?

A single `formData` object stores name, email, subject and message. `handleChange` uses the input's `name` attribute as a computed key: `{ ...formData, [e.target.name]: e.target.value }`. This avoids four separate `useState` calls.

Q99 What validations are applied?

All four fields are marked `required` and the email field uses `type="email"`, so the browser checks for a basic email pattern. There is no server-side validation, length limit, spam protection or rate limiting because there is no backend submission.

Q100 Are the displayed contact details real?

They are placeholders: `support@mentorlab.com`, `+91-9876543210` and "Mentor Lab, Education Hub, India". They should be replaced with verified contact information before publishing.

PAGE 3.5

Study Notes Hub

`/study-notes | Public | Static cards`

The Study Notes page is a simple course selector. It presents three cards, one for each course, and sends the student to that course's category page.

Q101 What is the purpose of the Study Notes page?

It acts as the entry point to the notes library. A student chooses BCA, Biotech or BBA and is taken to the corresponding syllabus/category page, where individual subjects and notes are listed.

Q102 Where do the three course links go?

BCA goes to `/bca`, Biotech goes to `/biotechsylla`, and BBA goes to `/bba`. Each destination uses the same coverflow category layout.

Q103 Is any API used on this page?

No. The three cards and their remote Unsplash images are hard-coded. Notes are fetched later, on the category page, using `GET /api/notes?course=...`.

Q104 What would you improve here?

Add a search box that queries notes across all courses, show a count of available notes per course, use local optimised images, and add a direct "Continue where you left off" section for logged-in students.

BCA Curriculum Page

`/bca | Public | Data: bcaSubjects.js`

The BCA page presents the three subject groups as an interactive coverflow carousel. The active card is in the centre, the next card on the right and the previous card on the left.

Sections on the page

Category	Subjects	Examples
Programming Languages	7	C, C++, Java, Python, HTML & Web Design, JavaScript, CSS
Logical Subjects	7	DBMS, DSA, Operating Systems, System Analysis, Computer Architecture, DAA, Networks
Mathematics Subjects	14	Calculus, Algebra, Geometry, Probability, Statistics, Correlation, Regression, Sampling

Q105 How does the coverflow carousel work?

The page keeps an `index` in state. For each category it calculates `offset = (i - index + n) % n`. Offset 0 gets class `cover-center`, offset 1 gets `cover-right`, and all others get `cover-left`. CSS positions and scales the cards based on those classes. The arrows call `go(index +/- 1)`, and the modulo expression wraps around at both ends.

Q106 Why is the modulo expression written as `((i % n) + n) % n` ?

JavaScript's `%` operator can return a negative result for negative inputs. Adding `n` before the second modulo keeps the result in the range `0..n-1`, so clicking "previous" on the first card wraps to the last card correctly.

Q107 Where does the category data come from?

`src/data/bcaSubjects.js` exports `bcaCategories` and a helper `getCategoryBySlug`. Each category has a slug, icon, name, description, image URL and an `items` array of subject names.

Q108 How does the user open a category?

Each card is a React Router `Link` to `/bca/${c.slug}`. Clicking any card, even a side card, opens that category. The route is handled by `BCACategoryPage`.

Q109 What does the "Subjects" count show?

It displays `c.items.length`, so it is computed from the actual data rather than hard-coded. Adding a subject to the array automatically updates the badge.

Q110 Does the BCA page fetch anything from the server?

No. It only loads the static category data and remote images. The note API is called on the category detail page, not here.

Q111 What are the responsive and accessibility concerns of the carousel?

On small screens the three-card layout can overflow; CSS must reduce card size or switch to a simple list. The arrow buttons have `aria-label` values, which is good, but keyboard users should also be able to focus and activate cards, and the animated transform should respect `prefers-reduced-motion`.

PAGE 3.7

BCA Category Page

`/bca/:categorySlug | Public | GET /api/notes?course=BCA`

This page lists every subject in the selected category, matches each subject to a note in the database, and shows Read Online, PDF or "Coming Soon". For the Programming Languages category it also renders the online Code Compiler.

Sections on the page

- **Header:** Back link, category name and description.
- **Detail panel:** icon, category title, subject count and a short hint.
- **Subject list:** each subject with a dot, name, note actions or a Coming Soon badge.
- **Code Compiler:** only when `categorySlug === "programming-languages"`.

Q112 How is the category selected from the URL?

`useParams()` reads `categorySlug`. The helper `getCategoryBySlug` searches the static BCA data. If no category matches, the page shows a "Category not found" message and a back link instead of crashing.

Q113 Which API does this page call?

`GET /api/notes?course=BCA`. The backend filters notes by the exact course string and excludes the large `content` field from the list response, returning lightweight metadata such as title, subject, `fileId`, `pdfUrl` and `hasContent`.

Q114 How is a note matched to a subject?

`getNoteForSubject` lowercases both the subject name from the static array and `note.subject`, then looks for an exact match. This means the admin must type the subject name exactly as it appears in the curriculum data; otherwise the page shows Coming Soon.

Q115 What is the difference between Read Online and PDF?

Read Online links to `/notes/:id`, which renders the extracted text as a styled web page. PDF links directly to `/api/notes/:id/pdf`, which streams the original file from GridFS with an inline content disposition. Read Online appears if the note has a file, a URL or web content; PDF appears only if a file or URL exists.

Q116 Why does the code compiler appear only on Programming Languages?

The compiler is useful for C, C++, Java, Python, JavaScript and HTML, which are all in that category. Showing it on Mathematics or Logical Subjects would add unnecessary weight and confuse the page's purpose.

Q117 How does the page handle a subject with no note?

It renders a `soon-badge` with the text "Coming Soon". This is a graceful empty state: the curriculum remains visible, and the absence of content is communicated without an error.

Q118 What happens if the notes API fails?

The `.catch` logs the error to the console and leaves `notes` as an empty array. Every subject then shows Coming Soon. A better UX would show an inline error message and a retry button, rather than silently implying that no notes exist.

`Q119` What are the limitations of the exact subject match?

Small differences in spelling, punctuation or spacing break the match. For example, "C++ Programming" and "C++" are different strings. A normalised match or a `subjectId` foreign key would be more reliable than free-text matching.

`Q120` How would you add a new BCA subject?

Add its name to the correct category's `items` array in `bcaSubjects.js`. Then upload a note through the admin dashboard with `course = BCA` and a `subject` string that exactly matches the new item. The page will automatically show Read Online/PDF.

PAGE 3.8

BBA Curriculum Page

`/bba | Public | Data: bbaCategories.js`

The BBA page uses the same coverflow pattern as BCA but with six business categories. It reuses `bca.css` and adds a BBA background class.

Sections on the page

Category	Focus
Management & Strategy	Management principles, strategy and organisational behaviour
Finance & Accounts	Accounting, financial management and cost concepts
Analytics & Tech	Business analytics, MIS and data-driven decisions
Marketing Dynamics	Marketing management, research and consumer behaviour
Environment & Law	Business law, ethics and the business environment
Operations & Practice	Operations, supply chain and practical business projects

`Q121` How is BBA different from BCA technically?

The implementation is almost identical: the same carousel logic, the same CSS file and the same route pattern. The differences are the data source (`bbaCategories`), the six categories instead of three, the heading text and the `bba-bg` class.

`Q122` Why does this page reuse `bca.css` ?

The card, coverflow and detail-panel styles are shared. Reusing the stylesheet keeps the visual language consistent and avoids duplicating CSS. A future refactor could rename it to something neutral such as `curriculum.css` .

`Q123` How many categories and subjects does BBA have?

There are six categories, and each category's subject count is computed from its `items` array rather than hard-coded. The count badge updates automatically if the data changes.

`Q124` What happens when a BBA card is clicked?

It navigates to `/bba/:categorySlug` , which is rendered by `BbaCategoryPage` . That page validates the slug and lists the subjects with the same note-matching pattern as BCA.

BBA Category Page

`/bba/:categorySlug | Public | GET /api/notes?course=BBA`

This page lists the subjects of a chosen BBA category, matches notes and offers Read Online/PDF links. It does not render the code compiler.

Q125 How does the BBA category page differ from the BCA category page?

It uses `bbaCategories` and queries `GET /api/notes?course=BBA`. It does not import `CodeCompiler`. The rest of the structure, the exact subject matching and the Coming Soon fallback are the same.

Q126 How does it handle an invalid slug?

If `getCategoryBySlug` returns nothing, it displays "Category not found" with a link back to `/bba`. This prevents a blank page or a runtime error when a user edits the URL.

Q127 Which notes appear on this page?

Only notes whose `course` field exactly equals `BBA` and whose `subject` exactly matches the subject name (case-insensitive). The list API deliberately excludes full content to keep the response small.

Q128 What is a possible improvement for all three category pages?

Extract the shared logic into one reusable `CourseCategoryPage` component that receives the course name, data source and an optional compiler flag. This would remove three near-duplicate files and reduce maintenance effort.

PAGE 3.10

Biotechnology Syllabus Page

`/biotechsylla | Public | Data: biotechCategories.js`

This is the Biotechnology equivalent of the BCA/BBA curriculum pages. It has three categories and a Biotech-specific background class.

Sections on the page

Category	Coverage
Foundation Sciences	Cell biology, chemistry, biostatistics, biomolecules, lab techniques
Core Biotechnology	Biochemistry, microbiology, molecular biology, enzymes, rDNA technology
Advanced & Applied	Genetic engineering, immunology, bioprocess, genomics, fermentation

Q129 What is the purpose of this page?

It organises the Biotechnology syllabus into three progressive categories and lets the student open a category to see its subjects, notes and PDFs. It is separate from the Biotech roadmap, which shows a six-phase career/learning path.

Q130 How does the page reuse existing code?

It follows the same carousel implementation as BCA and BBA, imports `bca.css`, and adds `biotech-bg` for the background. The data comes from `biotechCategories.js`. This consistency keeps the UI familiar.

Q131 What is the difference between the syllabus page and the Biotech roadmap?

The syllabus page is subject-oriented and links to study material. The roadmap at `/biotech` is a six-phase visual guide from foundation sciences to biopharmaceutical development, intended for planning and motivation rather than note navigation.

Q132 Where would a new Biotech category be added?

Add an object with `slug`, `icon`, `name`, `desc`, `image` and `items` to `biotechCategories.js`. The carousel and the route will pick it up automatically because the page maps over the array.

PAGE 3.11

Biotech Category Page

`/biotechsylla/:categorySlug | Public | GET /api/notes?course=BIOTECH`

This page lists the subjects of a selected Biotechnology category and matches them to notes stored with `course = BIOTECH`.

Q133 Which course value does the note API receive?

It queries `/api/notes?course=BIOTECH`. The backend performs an exact match on the `course` field, so notes uploaded as "Biotechnology" instead of "BIOTECH" will not appear. The admin upload form should enforce the correct course value.

Q134 How are note links shown?

If a matching note has a PDF or URL, the page shows Read Online and PDF. If it only has web content, it shows Read Online. If there is no matching note, it shows Coming Soon. This is the same logic as the other two category pages.

Q135 Why does this page not have a code compiler?

Biotechnology is not a programming course, so a compiler would not be relevant. The compiler is conditionally rendered only for BCA's Programming Languages category.

Q136 What is a data-consistency risk here?

The subject names in the static data and the subject names stored with uploaded notes must match exactly. A mismatch produces a false "Coming Soon". Using IDs or a normalised lookup would eliminate this risk.

PAGES 3.12 - 3.14

BCA, BBA and Biotech Roadmaps

`/roadmapbca` `/bbaroadmap` `/biotech` | Public | Static

Each course has a six-phase roadmap. The pages are static, visually rich and designed to show a student what to learn and when. They are separate from the syllabus pages, which are used to open notes.

Sections on each page

- **Header:** a step-by-step guide title for the course.
- **Phases 1-3:** foundation, core subjects and intermediate/web/advanced skills.
- **Phase 4:** specialisation or electives.
- **Phase 5:** projects, internship and practical experience.
- **Phase 6:** career, placement and alumni/graduation.
- **CTA:** a note linking back to the course's study notes.

Q137 What is the purpose of a roadmap page?

It gives students a structured, phase-wise plan rather than a flat subject list. It answers "what should I learn next?" and connects academic subjects with projects, specialisation and placement preparation.

Q138 How are the roadmap cards built?

They are static JSX sections. Each card has a phase tag, a focus sentence and a `<ul>` of subjects. CSS classes such as `card-blue`, `card-green` and `card-purple` give each card a different colour.

Q139 How many phases does each roadmap have?

All three have six phases. BCA moves from programming foundations to core computing, web/networks, AI specialisation, build/deploy and career. BBA moves from business foundations to core management, intermediate subjects, specialisation, practical experience and graduation. Biotech moves from foundational sciences to core biotech, advanced technical skills, bioinformatics, stem cells and biopharmaceutical development.

Q140 Why is Phase 4 called the elective phase?

It lets students choose a direction. BCA lists AI/ML, cyber security, cloud, mobile, data science and IoT; BBA lists finance, marketing, HR, entrepreneurship, analytics and supply chain. This mirrors how many degree programmes offer specialisation tracks.

Q141 Is the roadmap data stored in the database?

No. It is hard-coded in `roadmapbca.jsx`, `bbaroadmap.jsx` and `biotech.jsx`. This is fine for a fixed curriculum, but editing requires a code change and redeploy. A CMS or MongoDB collection would make it admin-editable.

Q142 What is the "Explore Study Notes" button for?

It links the roadmap back to the corresponding syllabus page: BCA to `/bca`, BBA to `/bba` and Biotech to `/biotechsylla`. This closes the loop between planning and actual study material.

Q143 Why does the project contain both `roadmapbca.jsx` and `BCARoadmap.jsx`?

`App.jsx` imports and routes `roadmapbca.jsx`. `BCARoadmap.jsx` is an unused duplicate of the same content. It should be removed to avoid confusion and dead code. This is a useful point to mention when asked about code quality.

`Q144` Are the roadmaps responsive?

They use CSS grid classes and a row divider. The layout can stack on smaller screens, but each roadmap has its own CSS file, so responsiveness must be checked separately for `roadmapbca.css`, `roadmapbb.css` and `bio.css`.

`Q145` What improvement would make the roadmaps more useful?

Add progress tracking: let a student tick off phases, store the progress in the User document, and show a completion percentage. Linking each subject to its note or mock test would turn the roadmap into an actionable learning path instead of a static poster.

Note Viewer

`/notes/:id | Public | GET /api/notes/:id`

The Note Viewer turns a PDF's extracted text into a clean reading page. It adds a table of contents, a reading-progress bar, reading-time metadata, language detection and a link to the original PDF.

Sections on the page

- **Reading progress bar:** fixed at the top, based on scroll position.
- **Article header:** back link, course badge, detected language, title, subject, description, word count and reading time.
- **Content area:** text converted into paragraphs, lists and headings.
- **Table of contents:** right-side unit list with active-section highlighting.
- **Footer:** PDF download link when the original file exists.

`Q146` How does the page load a note?

It reads the `id` from `useParams()` and calls `GET /api/notes/:id`. While waiting it shows "Loading note"; on failure it shows a "Note not found" panel with a link back to the notes hub.

`Q147` How is the note content rendered?

The backend stores extracted PDF text in the `content` field. The frontend passes it to `renderPlainText` in `utils/markdown.js`, which converts blank lines into paragraphs, bullet characters into lists, and heading-like lines into `h2` / `h3` elements. The result is inserted with `dangerouslySetInnerHTML`, but the renderer calls `escapeHtml` on every text fragment first, so stored text cannot inject raw HTML.

`Q148` Is the content actually Markdown?

No. Despite the file name `markdown.js`, it is a plain-text-to-HTML formatter with heuristics. It detects headings using patterns such as "Unit 1", "Chapter 2", numbered titles and all-uppercase lines. It does not support Markdown syntax like `**bold**` or links.

`Q149` How is the table of contents generated?

`extractHeadings` runs the same heading-detection logic and returns a list of `{ level, text }`. The TOC renders level-2 entries as units and level-3 entries as sub-items. Each heading receives an `id="nv-sec-i"`, and clicking a TOC item calls `scrollIntoView`.

`Q150` How does the page highlight the current section?

A scroll listener computes total progress and loops through the heading elements. The last heading whose top is above 120 pixels becomes the active index. The TOC then gives that item an active class and scrolls the TOC list so the active item stays centred.

`Q151` How is reading time calculated?

It counts words by splitting the content on whitespace and divides by 200 words per minute, with a minimum of one minute. The word count is also displayed as a metadata chip.

`Q152` How does language detection work?

`detectLanguage` applies regular expressions to the content. `#include <iostream>` or `std::cout` identifies C++; `public class` or `System.out.` identifies Java; `def` / `import` identifies Python; `#include <` or `printf` identifies C; `const` / `useState` identifies JavaScript; HTML tags identify Web. If nothing matches, no language badge is shown.

`Q153` Does opening a note mark it as read?

Yes, automatically. If a token exists, the page calls `POST /api/certificate/notes/:id/read`. The backend adds the note ID to the user's `readNotes` array using `$addToSet`, so duplicate reads do not create duplicates. The call is fire-and-forget; a failure does not block reading.

`Q154` How does the "Back" link know where to go?

A `COURSE_ROUTES` map converts the note's course to `/bca`, `/bba` or `/biotechsylla`. Unknown courses fall back to `/study-notes`. This keeps the navigation contextual.

`Q155` How is the PDF download served?

The link points to `/api/notes/:id/pdf`. The backend streams the GridFS file with the correct content type. In `index.js`, PDFs served from `/uploads` are set to `Content-Disposition: inline`; the notes route also streams inline, so the browser can preview the PDF in a new tab.

`Q156` What are the limitations of this page?

Heading detection is heuristic and can misclassify lines; long PDFs may produce a noisy TOC. The page marks a note as read immediately on open, even if the student does not scroll. There is no bookmark, highlight, font-size control or offline support. The content is escaped, which is safe, but it also means intentionally formatted HTML in a note will display as text.

PAGE 3.14

Recorded Lectures

`/recorded-lectures | Public | Static playlist`

The page is a simple video-learning screen with one main player and a static playlist sidebar.

Sections on the page

- **Header:** "Recorded Lectures" and the line "Watch and revise at your own pace."
- **Main player:** HTML5 video with the file `/intro2.mp4`.
- **Lecture information:** instructor, duration and upload date.
- **Course Playlist:** four HTML/CSS lectures with titles and durations.
- **Footer:** copyright line.

`Q157` Is the recorded-lecture content dynamic?

No. The single MP4 and all four playlist entries are hard-coded JSX. The playlist items are not clickable and do not change the video source. There is no lecture API, no progress tracking and no database model used on this page, even though the backend has an unused `Lecture` model.

`Q158` How is the video played?

It uses the browser's native `<video controls>` element with a single MP4 source from the public folder. The browser handles play, pause, seeking, volume and full screen; no custom video library is required.

`Q159` What is inconsistent on this page?

The footer says "Drona Gyan Platform", while the rest of the site uses "Mentor Lab", and the `package.json` name is `drona-gyan`. This suggests the project was renamed. The brand should be unified before a demo.

`Q160` How would you make the lecture page production-ready?

Store lectures in MongoDB or a video platform, make the playlist update the player, add course and chapter filters, track watched progress for signed-in students, support captions and playback speed, and move large videos to a CDN or object storage rather than the frontend public folder.

PYQ Portal

`/pyq | Public | GET /api/pyq?universityId=...`

The Previous Year Questions portal lets a student choose a university, then browse a course-wise list of subjects and download the uploaded PDF for each. It is a two-screen experience: university carousel and subject archive.

Sections on the page

- **University carousel:** six universities with image, city, category, description and a "View PYQs" button.
- **Auto-advance:** the carousel moves automatically when not hovered or animating.
- **University screen:** back button, heading, and three course columns (Biotech, BCA, BBA).
- **Subject list:** each subject with a View/Download button if a PDF exists, or "No PDF uploaded".

Q161 Which universities are included?

Ranchi University, Vinoba Bhave University (Hazaribagh), Dr. Shyama Prasad Mukherjee University (Ranchi), Binod Bihari Mahto Koyalanchal University (Dhanbad), BIT Mesra (Ranchi) and Sidhu Kanhu Murmu University (Dumka). Their IDs are `ranchi`, `vbu`, `dspmu`, `bbmku`, `bit` and `skmu`.

Q162 How does the university carousel work?

It keeps an `index`, calculates how many cards fit the viewport, and translates the card track with a CSS transform. It supports previous/next arrows, dot navigation, swipe gestures, hover pause and a timed auto-advance. Side cards get fade/scale classes for depth.

Q163 What happens after selecting a university?

`selectedUni` is set, the carousel view is replaced by the subject archive, and `fetchPYQs(universityId)` calls `GET /api/pyq?universityId=...`. The response is converted into a lookup map keyed by normalised course and subject.

Q164 How does the page match a PYQ to a subject?

`normalizeKey` lowercases the text, collapses whitespace and trims it. The map key is `course:subject`. When rendering a subject, the page builds the same key and looks up the PDF. This makes matching case- and spacing-insensitive, but punctuation and spelling still matter.

Q165 Where does the subject list come from?

It is a static `subjectsData` object with long arrays for `bca`, `bba` and `biotech`. The API only supplies the uploaded PDF records; it does not define the curriculum on this page.

Q166 How is a PYQ PDF opened?

The link points to `/api/pyq/:id/pdf`. The backend finds the record, opens a GridFS download stream from the `pyqs` bucket, sets the content type to PDF and streams it inline. If the file is missing, it returns an error status.

Q167 What happens for a subject with no uploaded paper?

The page shows a disabled "No PDF uploaded" state instead of a link. This is a clear empty state, although it can look like a large amount of missing content if the archive is new.

`Q168` How are PYQs uploaded?

An admin uses the dashboard's Upload PYQ section, choosing a university, course, subject and PDF. The form posts to `POST /api/pyq/upload` as multipart form data. Multer keeps the file in memory, and the backend writes it to the `pyqs` GridFS bucket with metadata.

`Q169` What are the limitations of the PYQ portal?

The university and subject lists are static, so adding a university requires a code change. Matching is by text rather than IDs. There is no search, no year/semester filter, no download count and no preview thumbnail. The upload route checks for a valid token but not specifically for an admin role.

Mock Test Portal

`/mock-test | Protected | /api/mocktest/*`

The mock-test module is the most interactive part of the project. It includes test selection, paid-test unlocking, a timed exam interface, a question palette, server-side scoring, a result scorecard and certificate checking.

Screens and sections

Screen	Main elements
Selection	Search box, test cards, subject/topic tags, question count, duration, price badge, start button
Payment modal	Amount, UPI ID, QR image, UTR input, Razorpay checkout or simulated confirmation
Exam	Full-screen layout, timer, question, four options, Save & Next, Mark for Review, Clear, palette, Submit
Result	Scorecard, attempted/correct/wrong counts, final score, review button, restart, certificate prompt

Q170 How does a student start a mock test?

The page calls `GET /api/mocktest` with the Bearer token. The backend hides `correctAnswer` from the list and marks each test with `purchased` or `paymentPending`. Clicking Start calls `openTest(test)`. If the test is free or already purchased, the exam screen opens immediately; otherwise the payment modal opens.

Q171 How is access to a paid test enforced?

The backend helper `hasAccess(userId, test)` returns true when the price is zero or when a Purchase document exists for that user and test. It is checked before fetching the full test, before submitting, and inside the order flow. The unique index on `userId + testId` prevents duplicate purchase records.

Q172 Which payment paths are supported?

Three paths: free tests, real Razorpay checkout when valid keys are configured, and a manual UPI fallback when Razorpay is not configured. The fallback shows the admin's UPI ID/QR, asks for the transaction reference (UTR), and calls `simulate-pay`. The purchase is stored with status `simulated`.

Q173 How is a real Razorpay payment verified?

The backend creates an order with amount `price * 100` (paise). After checkout, the frontend sends `orderId`, `paymentId` and `signature` to `POST /api/mocktest/:id/verify`. The server recomputes an HMAC-SHA256 of `orderId|paymentId` with the Razorpay secret and compares it to the signature. Only a match creates a `paid` purchase.

Q174 What is the purpose of the UTR field?

In the manual UPI flow, the student pays outside the app and enters the UPI transaction reference. The server requires a non-empty reference for a paid test and stores it as `paymentId`. This creates an audit trail for the admin, although the server cannot automatically prove that the UTR is genuine.

Q175 How does the exam timer work?

When a test starts, `timeLeft` is set to `(test.duration || 60) * 60` seconds. A `setInterval` decrements it every second while the test screen is active. When it reaches zero, `handleSubmit(true)` is called automatically. The final submit always happens on the server.

`Q176` What do the question-palette colours mean?

Green means answered, red means not answered, yellow means marked for review. The palette also highlights the current question. The student can jump to any question by clicking its number, and the status of the previous question is updated when moving away.

`Q177` What is the difference between "Save & Next", "Mark for Review" and "Clear"?

Save & Next records the selected option and advances. Mark for Review keeps the question flagged for a later check. Clear removes the selected answer and marks the question as not answered. These actions only change local React state until the final submit.

`Q178` How is the test scored?

On the server, each attempted question is compared with its stored correct answer. The score is `correct * 4 - wrong * 1`, and the maximum is `questions.length * 4`. Unattempted questions add zero. There is no floor, so a test with many wrong answers can produce a negative score.

`Q179` Why is scoring done on the server instead of the browser?

Server-side scoring prevents a student from editing JavaScript to inflate the result or reading the answer key from the page. The list endpoint removes `correctAnswer`, and the submit endpoint recomputes the score from the database copy of the test.

`Q180` What happens after a test is submitted?

The server creates a `Result` document with the answers, counts and score, and returns `correct`, `wrong`, `score`, `totalQuestions`, `maxScore` and `resultId`. The frontend also posts to `/api/auth/add-activity` to add a recent-activity entry and increment the tests-taken count. It then asks the certificate endpoint whether the course is complete.

`Q181` How does the result screen show the certificate?

After submission, the page calls `GET /api/certificate/progress/:subject` using the test's subject. If the course is complete and a certificate exists or was just issued, the result screen shows a "Congratulations - Course Completed!" card with a link to the certificate.

`Q182` How does a student review answers?

The result screen has a Review button that navigates to `/test-review/:resultId`. The review page fetches the saved result and the associated questions, then shows the selected answer, the correct answer and the status for each question.

`Q183` What happens if the student refreshes during a test?

The exam state is held in React state and is not persisted. A refresh returns the student to the selection screen and loses the in-progress answers. A production exam system would save answers to the server or local storage periodically.

`Q184` What are the main limitations of the mock-test module?

There is no negative-score floor, no section-wise timing, no randomised question order, no attempt limit, no server-side autosave and no anti-cheating/tab-switch detection. The manual UPI path trusts the entered UTR. The result review shows a small icon bug: a correct selected answer also displays a cross icon because the "Your Answer" label uses a cross for both cases.

PAGE 3.17

Test Review Page

`/test-review/:resultId | Protected | GET /api/mocktest/result/:resultId`

The review page is a detailed answer sheet for one completed test. It is reached from the result screen or from a Profile activity item.

Sections on the page

- **Header:** back button, "Answer Review: [test title]" and a divider.
- **Summary row:** score, attempted, correct, wrong and unattempted.
- **Question cards:** each question, all options, correct answer highlighting, selected answer highlighting and a status badge.
- **Footer action:** "Back to Mock Tests".

Q185 Which API does the review page use?

`GET /api/mocktest/result/:resultId` with the Bearer token. The backend finds the result by ID and also filters by `userId: req.user._id`, so one student cannot open another student's result by guessing the ID.

Q186 How does it combine answers with questions?

The response contains the saved `result` and the current `questions` from the test. For each question, the page finds the matching entry in `result.answers` by `questionIndex`, then uses its `selectedOption` and `isCorrect` values to style the options.

Q187 How are the three states displayed?

A question is **correct** if a selected option matches the correct answer, **wrong** if an option was selected but does not match, and **unattempted** if `selectedOption` is -1 or missing. The card border and badge colour change for each state.

Q188 What happens if the result does not exist?

The page catches the error and shows the server message with a "Go Back" button. It does not crash or show an empty review card.

Q189 What small bug is visible in the review UI?

When the selected option is also the correct option, the code adds both "Correct Answer" and a cross-marked "Your Answer" tag. The second tag should use a check mark or a neutral "Your Answer" label. It is a presentation bug, not a scoring bug.

Performance Analytics

`/analytics | Protected | GET /api/auth/profile`

The analytics page turns the student's stored activity into visual feedback. It has four tabs: Overview, Score Trends, Subject Analysis and Test History.

Sections on the page

- **Overview:** six stat cards (Tests Taken, Accuracy, Rank, Day Streak, Hours Learnt, Avg Score), recent score trend and subject performance.
- **Score Trends:** score progression chart for the last 10 tests.
- **Subject Analysis:** per-course performance and monthly activity.
- **Test History:** the full recent-activity list with scores and review links.
- **Insights:** latest trend and a recommendation based on the data.

Q190 Where does the analytics data come from?

It comes from `GET /api/auth/profile`, specifically the `performanceStats` object and the `recentActivity` array. There is no separate analytics API; the calculations are done in the React component.

Q191 How is accuracy calculated?

Each activity has a score string such as "24 / 100". `parseScore` splits it into obtained and total marks. `calculateAccuracy` sums all obtained marks and all maximum marks, then returns `obtained / total * 100`, rounded to a whole number.

Q192 How is the score trend built?

It takes the last 10 activities, converts each score to a percentage and returns a labelled bar for each test. The chart is CSS-based rather than a chart library: each bar's height is set to the percentage value.

Q193 How is subject performance inferred?

It inspects the activity title for the words "BCA", "BBA" or "biotech" and groups the scores into those buckets. If none match, the activity goes into "General". This is a heuristic, not a real subject field.

Q194 How is the day streak calculated?

It builds a set of unique activity dates, sorts them newest first, and walks backwards from today. Each date within one day of the previous date increases the streak; a gap stops the count. Multiple activities on the same day count once.

Q195 Which stat cards may not update as expected?

`addActivity` increments only `performanceStats.testsTaken`. It does not update `avgScore`, `rank` or `hoursLearnt`, so those cards can remain at their defaults even after several tests. Accuracy, streak and trends are computed from activities and do update.

Q196 What are the limitations of the analytics page?

It relies on activity titles for subject classification, uses a score string instead of numeric fields, has no date-range filter, and does not compare the student with other students for a real rank. A dedicated analytics endpoint with aggregated result data would be more reliable.

Student Profile

`/profile | Logged-in student | GET/PUT /api/auth/profile`

The profile page shows identity, college information, performance statistics, certificate count and recent test activity. It also allows the student to edit the college name.

Sections on the page

- **Header:** initials avatar, role badge, full name, email, student ID and college.
- **College editor:** add or edit the college, with save/cancel and a success message.
- **Sidebar:** performance overview and earned-certificate count.
- **Main panel:** recent test activity with score and Review button.

Q197 How does the page load data?

Two requests run in parallel: `GET /api/auth/profile` for the user document and `GET /api/certificate/mine` for the certificate list. The certificate count is the length of the returned array. If the profile fails, the page shows an error; if certificates fail, the count simply becomes zero.

Q198 What can the student edit?

Only the college name. The save handler sends `PUT /api/auth/profile` with `{ college }`. The backend also accepts `name` and `studentId`, but the UI does not expose them.

Q199 How is the role badge decided?

It reads `user.isAdmin` from the profile response: true shows "ADMINISTRATOR", false shows "STUDENT". This is display only; it does not grant any permissions.

Q200 Why can the profile page still load for an admin?

Admins are stored in the same User collection and have a valid token, so the profile API returns their record. The page labels them as an administrator and shows their activity, which may be empty.

Q201 How does the Review button in recent activity work?

Each activity stores a `resultId` when it comes from a mock-test submission. If present, the button navigates to `/test-review/:resultId`; otherwise the button is not rendered.

Q202 Is the profile route protected?

`App.jsx` does not wrap `/profile` in `ProtectedRoute`. Instead, the page calls the protected profile API with whatever token exists. Without a valid token the API returns 401, and the page shows "Failed to load profile". Adding the route guard would give a cleaner redirect to login.

Q203 What is missing from the profile page?

There is no avatar upload, no password-change form, no email or phone editing, no account-deletion option and no direct edit of name/student ID. The "College saved" message is partly in Hinglish, which should be localised consistently.

PAGES 3.20 - 3.21

My Certificates and Certificate View

`/certificates` `/certificate/:id` | Protected | `/api/certificate/*`

The certificate module lists a student's earned certificates and renders a printable certificate of course completion.

Sections on the page

- **My Certificates:** heading, explanatory text, empty state, certificate cards with course, ID and issue date.
- **Certificate View:** action bar, decorative certificate, brand, student details table, completion statement, certificate ID and date, signature line.
- **Print:** a "Print / Save as PDF" button that calls `window.print()`.

Q204 How does a student earn a certificate?

The backend's `getCourseProgress` counts all notes for the course and all mock tests whose subject matches the course. The course is complete only when there is content and the student has read every note and attempted every test. `GET /api/certificate/progress/:course` checks this, creates the certificate if needed, and returns it.

Q205 How are course names normalised?

`normalizeCourse` maps aliases to `BCA`, `BBA` or `BIOTECH`. For example, "BIOTECHNOLOGY" and "BIO" both become `BIOTECH`. Notes are matched by a case-insensitive regular expression, and tests are matched by subject patterns such as `/BIOTECH|BIOTECHNOLOGY|\bBIO\b/i`.

Q206 How is the certificate ID generated?

It uses the format `ML-YYYYMMDD-XXXXXXXX`: the current date plus eight random hexadecimal characters from `crypto.randomBytes(4)`, converted to uppercase. The field is unique in the database.

Q207 What data is copied onto the certificate?

Student name, college, phone, email, student ID, course, issue date and certificate ID. The page prefers values stored on the certificate and falls back to the current profile for any blank field, so updating a college later can still appear if the certificate did not store it.

Q208 Can a student view another student's certificate?

No. `GET /api/certificate/:id` looks up the certificate by ID and checks `userId === req.user._id`. A mismatch returns an error, and the page shows "Certificate not found".

Q209 How is the certificate downloaded as a PDF?

There is no server-side PDF generator. The page is styled for print, and the button calls `window.print()`; the student then chooses "Save as PDF" in the browser's print dialog. This is simple and reliable, but the layout depends on the browser's print settings.

Q210 Can duplicate certificates be created?

The progress endpoint checks for an existing certificate with the same `userId` and `course` before creating one. The unique `certificateId` also prevents duplicate IDs. However, there is no compound unique index on `userId + course`, so two simultaneous requests could theoretically race; a compound index would close that gap.

`Q211` **What are the limitations of the certificate feature?**

Completion is all-or-nothing: every note and every test in the course must be completed. There is no public certificate verification page, no QR authenticity check, no authorised signature image and no server-generated PDF. The "Back to Profile" link on the certificate page is labelled "My Certificates" but goes to `/profile`, which is a minor navigation inconsistency.

AI Chatbot and Full-Page Assistant

`Floating widget on every page + /chatbot`

Mentor Lab includes a floating AI assistant and a full-screen version of the same chat. It supports English and Hindi, text-to-speech playback, a typing indicator and a shortcut to the full-page experience.

Sections on the page

- **Floating launcher:** bottom-corner button that opens a compact chat window.
- **Chat header:** title "Mentor Lab Assistant", language selector, expand and close buttons.
- **Messages:** user and bot bubbles, welcome message, typing indicator.
- **Composer:** text input and send button.
- **Speak control:** per-message playback using the browser voice or server TTS.
- **Full page:** the same chat with a larger layout, timestamps and suggestion prompts.

`Q212` Which AI service powers the chatbot?

The backend calls the Groq OpenAI-compatible chat-completions API at `https://api.groq.com/openai/v1/chat/completions` with the model `openai/gpt-oss-20b` and the server's `GROQ_API_KEY`. The frontend never receives the key.

`Q213` What is the chatbot's system prompt?

It positions the model as a friendly college guidance assistant for Mentor Lab, specialising in BCA, BBA and Biotech courses, exams and careers. It instructs short, direct, conversational answers of 2-4 sentences, no filler, and a strict language choice: English or Hindi in Devanagari only.

`Q214` How is the language selected?

The chat window has an English/Hindi selector. The value is sent as `language` in the request body. The backend adds a language instruction to both the system prompt and the user message, so a Hindi request is answered in Hindi.

`Q215` Does the chatbot remember previous messages?

No. Each request sends only the current message and language. The server does not receive the conversation history, so follow-up questions that depend on context may be misunderstood. Adding a message array or a session ID would enable real multi-turn conversation.

`Q216` How does text-to-speech work?

For English, the browser's `speechSynthesis` API reads the reply locally. For Hindi, the frontend splits the text into chunks of about 180 characters and sends each chunk to `POST /api/ai/tts`. The backend proxies Google Translate TTS, returns `audio/mpeg`, and the browser plays the chunks in order. It falls back to the browser's Hindi voice if needed.

`Q217` Why are the text chunks limited to 180 characters?

Google Translate's TTS endpoint has a practical request-length limit. Chunking at word boundaries avoids truncated audio and keeps each request small. The browser plays the chunks sequentially with `onended` callbacks.

`Q218` What is the `/explain` endpoint used for?

It is intended to explain a note or topic. It accepts a subject, course and language, tries to find a local PDF named after the subject, extracts up to 8,000 characters, and asks Groq for a simple explanation ending with key points. The frontend contains an `AIExplainModal` component for this, but it is not imported by any page, so the feature is currently backend-only.

`Q219` What is the biggest deployment bug in the chatbot?

Both `Chatbot.jsx` and `ChatbotPage.jsx` call `http://localhost:5000/api/ai/chat` and `http://localhost:5000/api/ai/tts` directly instead of using a relative `/api` path. This works only on the developer's machine; after deployment the browser will try to reach the visitor's own localhost. The URLs should be relative or read from an environment variable.

`Q220` Is the AI API authenticated or rate-limited?

No. `/api/ai/chat`, `/explain` and `/tts` are public and have no per-user quota or rate limiter. This exposes the Groq key's quota to abuse. Authentication, request throttling and a maximum message length would be appropriate before public deployment.

`Q221` What are the security and privacy considerations of the AI feature?

Student messages are sent to a third-party AI provider, so the privacy policy should disclose this. The backend must never expose the Groq key. Prompt injection is possible because user text is passed directly, so the system prompt should be treated as guidance, not a security boundary. TTS also depends on an unofficial Google endpoint, which may change or block traffic.

PAGE 3.23

Online Code Compiler

`Embedded on /bca/programming-languages | Public`

The Code Compiler is a browser-based editor that lets a student write and run code without installing an IDE. It is embedded only in the BCA Programming Languages category.

Sections on the page

- **Header:** "Online Code Compiler" and a short description.
- **Language tabs:** C, C++, Java, Python, JavaScript and HTML/CSS/JS.
- **Editor:** a monospace textarea with a language-specific starter template.
- **Standard input:** optional stdin box for programs that read input.
- **Run button:** compiles and executes the code.
- **Output panel:** stdout, compiler errors or runtime errors; HTML mode shows an iframe preview.

`Q222` Which compiler service does the component use?

It sends a POST request to `https://wandbox.org/api/compile.json` with `{ code, compiler, stdin }`. Wandbox is a public online compilation service. The compiler IDs are `gcc-12.3.0-c` for C, `gcc-12.3.0` for C++, `openjdk-jdk-21+35` for Java, `cpython-3.11.10` for Python and `nodejs-20.17.0` for JavaScript.

`Q223` How does the component handle HTML/CSS/JS?

HTML has no compiler ID. When Run is clicked, the code is placed directly into an `<iframe sandbox="allow-scripts" srcDoc={code}>`. The sandbox allows scripts but blocks same-origin access, which reduces the risk of the preview interfering with the parent page.

`Q224` How are output and errors displayed?

The component reads `program_output` for normal output. If `compiler_error` exists, it shows that; otherwise it checks `program_error`. If there is no output, it displays "(no output)". The output panel turns into an error style when an error is present.

`Q225` What happens when the student switches language?

`selectLanguage` updates the selected language, replaces the editor content with that language's starter template, and clears the output, error and HTML preview. This prevents Python code from being submitted as Java.

`Q226` Is the compiler integrated with the backend?

No. The browser calls Wandbox directly. This keeps the backend simple, but it means the feature depends on a third-party service, is subject to CORS/network failures, and cannot be rate-limited or audited by the application server.

`Q227` Can a student save or submit code?

No. Code lives only in React state. There is no save, no download, no submission history and no link to a mock-test question. Refreshing the page resets the editor to the default Python template.

Q228 **What are the limitations and risks of this compiler?**

It requires an internet connection, depends on Wandbox availability and quotas, has no execution timeout shown to the user, and the HTML preview can run arbitrary JavaScript in a sandboxed iframe. For a real product, a self-hosted sandboxed runner (for example Judge0 or Piston) with CPU/memory/time limits would be safer and more controllable.

Admin Dashboard

`/admin | AdminRoute + server checks | /api/admin, /api/notes, /api/pyq, /api/payment`

The dashboard is the control centre for content, students, tests and payments. It is a single-page layout with a fixed sidebar and seven sections.

Sections on the page

Section	What the admin can do
Overview	Stat cards and charts: platform overview, content distribution, registration trend, tests and questions by subject
Mock Tests	Upload a PDF to auto-create a test, or add questions manually; set title, subject, topic, duration and price
Students	View registered students with name, email, student ID and registration date
Purchases	View all purchases, manually unlock a test for a student, confirm a pending payment
Payment Settings	Set UPI ID, phone, payee name and upload a QR image; preview the current settings
Upload Notes	Upload a PDF with title, description, course and subject; view and delete notes
Upload PYQ	Upload a PYQ PDF with university, course and subject; view and delete papers

`Q229` How is the admin dashboard protected?

The route is wrapped in `AdminRoute`, which requires a token and `isAdmin === "true"` in local storage. The sensitive write actions also send the token, and some server routes check `req.user.isAdmin`. However, several read-only admin endpoints such as `/api/admin/stats` and `/api/admin/students` currently have no `protect` or admin middleware, so this should be fixed.

`Q230` Which API provides the dashboard statistics?

`GET /api/admin/stats`. It returns counts of user accounts (including admins), courses, notes, lectures, tests and total questions, plus a subject breakdown and a six-month student-registration trend. The frontend renders these with Recharts pie/bar/line charts and stat cards.

`Q231` How does the admin add a mock test from a PDF?

The admin chooses "Upload PDF", fills in title, subject, topic, duration and price, and selects a PDF. The form posts to `POST /api/mocktest/upload-pdf`. The server stores the file, extracts text with pdf-parse, parses questions, options and answers, and creates a MockTest document. The PDF should follow a format such as "1. Question", "A) Option" and "Ans: A".

`Q232` How does manual test creation work?

The admin chooses "Add Manually", enters the test metadata and one or more questions, each with four options and a correct answer. The form posts to `POST /api/mocktest/add`. The frontend validates that at least one question has text and all four options filled before submitting.

`Q233` How is a test unlocked manually?

In the Purchases section, the admin selects a student and a test and submits `POST /api/admin/purchase/manual`. The route checks `req.user.isAdmin`, verifies that the test exists, and upserts a Purchase with status `manual`, amount equal to the test price and placeholder order/payment IDs.

`Q234` How does the admin confirm a pending payment?

The admin submits the purchase ID to `POST /api/admin/purchase/confirm`. The route verifies the admin role and updates the purchase status to `simulated`, which makes `hasAccess` return true. This is intended for manual UPI reconciliation.

`Q235` How are payment settings stored?

They are stored in a single Setting document with key `payments`, containing UPI ID, phone, payee name and QR image path. The admin reads and writes through `GET/POST /api/payment/settings/admin`. The public `GET /api/payment/settings` is used by the student payment modal.

`Q236` How is the payment QR image handled?

The admin form uses Multer with disk storage and uploads the image to the backend `uploads` folder with a generated name such as `qr_*.png`. The old QR file is deleted when a new one is saved. The public settings response exposes a QR URL for the student modal.

`Q237` How does the admin upload study notes?

The dashboard form collects course, subject, title, description and a PDF. It posts multipart data to `POST /api/notes/upload`. The backend also supports plain web content, but the dashboard exposes only the PDF path. A PDF is stored in the `pdfs` GridFS bucket, its text is extracted with pdf-parse, and the note is saved with `fileId`, `content` and `hasContent`.

`Q238` How does the admin upload a PYQ?

The form collects university, course, subject and a PDF and posts to `POST /api/pyq/upload`. The file goes into the `pyqs` GridFS bucket, and a PYQ document stores the university ID/name, course, subject and file ID. The student portal then finds it by university ID and course/subject key.

`Q239` How does the admin delete notes or PYQs?

The dashboard calls `DELETE /api/notes/:id` or `DELETE /api/pyq/:id` with the token and refreshes the list. The routes require authentication but do not explicitly check `isAdmin`, so a logged-in student could call them directly. This should be hardened with an admin middleware.

`Q240` What are the main limitations of the admin dashboard?

Some read endpoints are public, role checks are inconsistent across modules, there is no pagination for large student/purchase lists, no search/filter on most tables, and no audit log. The UI is a large 1,000+ line component, so splitting each section into its own component would improve maintainability. Real Razorpay configuration and email templates are not managed from the dashboard.

PART 04

04

Backend, API and Database Reference

Technical reference for the viva: server startup, request flow, Mongoose models and the complete endpoint map.

4.1

Server Startup and Request Pipeline

Q241 How does the backend start?

`backend/src/index.js` loads environment variables with `dotenv`, creates the Express app, enables `cors()` and `express.json()`, mounts the eight route modules under `/api`, serves the `uploads` folder, serves the built frontend from `frontend/dist`, calls `connectDB()`, and starts listening on `PORT` (default 5000). MongoDB connection is therefore initiated at startup, while the HTTP server can begin listening before the connection has fully completed.

Q242 What is the full request lifecycle for an API call?

Browser → Vite dev proxy or production same-origin request → Express → CORS and JSON middleware → route module → optional `protect` / `isAdmin` middleware → controller → Mongoose model → MongoDB → JSON response. In development, Vite proxies `/api` and `/uploads` to `http://localhost:5000`, so the frontend can use relative URLs without CORS problems.

Q243 How is the production frontend served?

After the API routes, Express serves `frontend/dist` as static files. A catch-all route `app.get("/{*path}")` returns `index.html`, which allows React Router to handle deep links such as `/mock-test` or `/notes/123`. A build must exist in `frontend/dist`; otherwise the catch-all points to a missing file.

Q244 What happens to an unknown `/api` URL?

Because the catch-all is registered after the API routers but matches every path, an unknown `/api/...` request can fall through to `index.html` instead of returning a JSON 404. This can confuse API clients. A dedicated `app.use("/api", ...)` 404 handler before the SPA fallback would make the behaviour correct.

Q245 Is there a global error handler?

No. Controllers use `try/catch` and return their own status codes and messages. Multer errors, malformed JSON, unexpected promise rejections and database connection failures are not normalised by a central middleware. A production app should add one error handler with consistent JSON shape and logging.

Q246 How is CORS configured?

`app.use(cors())` allows every origin. This is convenient in development but too permissive for production, especially with token-based APIs. It should be restricted to the deployed frontend origin through an environment variable.

Q247 How are files stored?

Two storage strategies are used. PDFs for notes and PYQs go into MongoDB GridFS buckets (`pdfs` and `pyqs`) using `multer.memoryStorage()`, so the file never needs a permanent disk path. Mock-test PDFs and the payment QR image use disk storage under `backend/uploads` with generated filenames. The uploads folder is served statically.

Q248 Why use GridFS instead of saving PDFs on disk?

GridFS keeps the file and its metadata in MongoDB, survives redeploys without a persistent disk, supports files larger than 16 MB by chunking, and lets the database record be deleted together with its file ID. The trade-off is extra database load and the need to clean up orphaned chunks manually.

`Q249` How are PDFs streamed to the browser?

The route opens a GridFS download stream by file ID, sets `Content-Type: application/pdf` and pipes the stream to the response. The static uploads middleware also sets `Content-Disposition: inline` for PDFs, so the browser previews rather than force-downloads them.

4.2

Database Models

Model	Purpose and key fields
User	Student and admin accounts: firstName, middleName, lastName, full name, unique email, password hash, phone, student ID, college, isAdmin, reset fields, performanceStats, recentActivity, readNotes, certificate IDs.
Note	Study material: title, description, course, subject, type (pdf/web), extracted content, GridFS fileId, optional pdfUrl, hasContent flag.
PYQ	Previous-year paper: universityId, universityName, course, subject, GridFS fileId, optional pdfUrl.
MockTest	Test metadata and questions: title, subject, topic, duration, price, createdAt, array of question objects (question, four options, correctAnswer).
Result	One submitted attempt: userId, testId, title, per-question answers, score, maxScore, attempted, correct and wrong counts, timestamps.
Purchase	Test entitlement: userId, testId, amount, status (pending/paid/simulated/manual), orderId, paymentId; unique compound index on userId + testId.
Certificate	Course completion record: userId, course, subject, unique certificateId, copied student details, score, issue date.
Setting	Key-value configuration; used for the public payment/UPI settings document.
Lecture	Legacy model for recorded lectures. The Recorded Lectures page does not use it; its content is static.

Q250 Why is the password field marked `select: false` ?

Mongoose excludes it from normal queries, so accidentally returning a user object does not leak the bcrypt hash. Login explicitly uses `.select("+password")` when it needs to compare the password.

Q251 What is stored in `performanceStats` ?

Tests taken, average score, rank and hours learnt. Tests taken is incremented by the add-activity endpoint; the other fields are present in the schema but are not automatically recomputed from results, which is why analytics calculates accuracy and trends from `recentActivity` instead.

Q252 What is stored in `readNotes` ?

An array of Note ObjectIds. `POST /api/certificate/notes/:id/read` adds the note with `$addToSet`, which prevents duplicates. Certificate progress converts the array to a set, filters the course's notes against it, and compares the count with the course's total notes.

Q253 How does the unique purchase index work?

A compound unique index on `{ userId, testId }` guarantees that one student can own at most one purchase record per test. The order flow uses `findOneAndUpdate` with `upsert`, so retrying a payment updates the existing record instead of creating duplicates.

Q254 Why does Result store the answers as an array instead of only a score?

The array preserves which option the student chose for each question, which is required by the Test Review page. It also makes the result auditable if a test's answer key is corrected later, although the current review page uses the stored `isCorrect` flags.

4.3

API Endpoint Map

Method and path	Purpose	Access
POST `/api/auth/register`	Create student account	Public
POST `/api/auth/login`	Student or admin login	Public
POST `/api/auth/forgot-password`	Request an OTP	Public
POST `/api/auth/verify-otp`	Verify OTP and receive reset token	Public
POST `/api/auth/reset-password`	Set a new password with reset token	Public
GET/PUT `/api/auth/profile`	Read or update profile	Token
POST `/api/auth/add-activity`	Record a test activity and increment tests taken	Token
GET `/api/notes` , `/api/notes/:id`	List notes or read one note	Public
GET `/api/notes/:id/pdf`	Stream the note PDF	Public
POST `/api/notes/upload`	Create a PDF or web note	Token (admin role not checked)
DELETE `/api/notes/:id`	Delete a note and its GridFS file	Token (admin role not checked)
GET `/api/pyq?universityId=`	List PYQs for a university	Public
GET `/api/pyq/:id/pdf`	Stream a PYQ PDF	Public
POST `/api/pyq/upload`	Upload a PYQ	Token (admin role not checked)
DELETE `/api/pyq/:id`	Delete a PYQ	Token (admin role not checked)
GET `/api/mocktest`	List tests without correct answers	Token
GET `/api/mocktest/:id`	Get one test, access-checked	Token + access
POST `/api/mocktest/:id/submit`	Submit answers and receive score	Token + access
POST `/api/mocktest/upload-pdf`	Parse a PDF and create a test	Token (admin role not checked)
POST `/api/mocktest/add`	Create a test manually	Token (admin role not checked)
POST `/api/mocktest/:id/order`	Create a Razorpay order	Token
POST `/api/mocktest/:id/verify`	Verify Razorpay signature	Token
POST `/api/mocktest/:id/simulate-pay`	Manual UPI fallback	Token
GET `/api/mocktest/results/mine`	List the student's results	Token
GET `/api/mocktest/result/:resultId`	Review one owned result	Token + owner
GET `/api/certificate/progress/:course`	Check progress, issue certificate	Token
POST `/api/certificate/notes/:id/read`	Mark a note as read	Token

GET `/api/certificate/mine`	List owned certificates	Token
GET `/api/certificate/:id`	View one owned certificate	Token + owner
POST `/api/ai/chat`	Ask the AI assistant	Public
POST `/api/ai/explain`	Explain a topic from notes	Public
POST `/api/ai/tts`	Hindi text-to-speech proxy	Public
GET `/api/payment/settings`	Public UPI settings	Public
GET/POST `/api/payment/settings/admin`	Read/update payment settings	Token
GET `/api/admin/stats`	Dashboard statistics	Currently public
GET/POST `/api/admin/courses`	Course list / create course	Currently public
GET `/api/admin/students`	Student list	Currently public
GET `/api/admin/purchases`	Purchase list	Token + admin
POST `/api/admin/purchase/manual`	Manually unlock a test	Token + admin
POST `/api/admin/purchase/confirm`	Confirm a pending purchase	Token + admin

> `Q255` **What is the standard response shape?**
>
> There is no enforced envelope. Successful endpoints usually return the resource directly or an object such as `{ message, test }`; errors return `{ message }` with 400, 401, 403, 404 or 500. A consistent shape such as `{ success, data, error }` would make the frontend simpler.

> `Q256` **How does the frontend attach the token?**
>
> Each page reads `localStorage.getItem("token")` and sets an `Authorization: Bearer <token>` header with Axios or `fetch`. There is no global Axios interceptor, so every protected call must remember to add the header; an interceptor would remove this duplication.

PART 05

05 Feature Deep Dives

The flows an examiner is most likely to ask you to trace end-to-end, written as compact whiteboard sequences.

5.1

Student Registration and Login

Q257 Trace registration from form to database.

1) The student submits name, email, phone, student ID, college and password. 2) The frontend validates required fields and local password rules. 3) `POST /api/auth/register` trims and validates the name, checks phone length for supported countries, checks the email domain against a free/fake-domain list, and rejects a duplicate email. 4) The password is hashed with bcrypt (10 rounds). 5) The user is created with `isAdmin: false`. 6) A JWT is signed for 7 days. 7) The response contains the user and token; the frontend stores both and logs the student in.

Q258 Trace login.

1) `POST /api/auth/login` receives email and password. 2) The user is found by email with `.select("+password")`. 3) If no user or no password match, a generic "Invalid email or password" is returned. 4) A JWT is signed with `{ id: user._id }`. 5) The frontend stores the token and a sanitised user object. 6) `AdminLogin` performs the same call but rejects the response if `isAdmin` is not true.

Q259 How does a protected request prove identity?

The `protect` middleware reads the Bearer token, verifies it with `JWT_SECRET`, loads the user by the decoded ID, attaches the user to `req.user` and calls `next()`. A missing or invalid token returns 401. Because the user is reloaded from MongoDB, deleting or demoting a user takes effect on the next request.

Q260 How does password reset work?

1) Forgot-password finds the user and enforces a 60-second resend cooldown. 2) A six-digit OTP is generated with `crypto.randomInt`, hashed with bcrypt, saved with a 10-minute expiry and an attempt counter, and emailed. 3) Verify-otp compares the hash, increments attempts on failure, and after success returns a short-lived reset JWT (10 minutes) whose ID is stored as a SHA-256 hash. 4) Reset-password verifies that JWT, checks the strong-password rules, hashes the new password and clears the reset fields.

Q261 What are the main weaknesses in the auth flow?

Signup does not apply the same server-side strong-password check as reset; email verification is absent; there is no rate limiting on login or signup; tokens live in `localStorage`; and email casing is not normalised consistently, so `User@x.com` and `user@x.com` can be treated as different accounts depending on the path. These are concrete improvements to mention.

5.2

Mock Test: Selection to Score

Q262 Trace a free test attempt.

1) `GET /api/mocktest` returns tests without answer keys. 2) The student clicks Start; `openTest` sees `price === 0` and loads the full test. 3) The exam screen initialises answers to -1 and starts the timer. 4) On submit, `POST /api/mocktest/:id/submit` receives an answers array. 5) The server recomputes correctness and score, creates a Result, and returns the result ID and summary. 6) The frontend records activity and shows the scorecard.

Q263 Trace a paid test attempt.

1) The test card shows a price and a lock. 2) Start opens the payment modal. 3) If Razorpay is configured, the frontend requests an order and opens checkout; otherwise it shows the UPI QR and UTR field. 4) The backend verifies the Razorpay signature or accepts the simulated UTR and creates a Purchase. 5) `hasAccess` now returns true. 6) The student starts the test and the normal submit flow runs.

Q264 Where exactly is the score calculated?

Inside the submit controller. It iterates over the test's questions, resolves the selected answer text with `q.options[answers[idx]]`, compares that text with `q.correctAnswer`, increments correct or wrong, and computes `score = correct * 4 - wrong`. It then stores the per-question result and the totals. The browser only displays what the server returns.

Q265 Can a student call the submit endpoint twice?

Yes. There is no unique constraint on `user + test` in the Result model and no attempt limit, so each submit creates a new Result. This may be desirable for practice, but it means "best score" or "first attempt" logic must be defined if the product needs it.

5.3

Payment State Machine

State	Meaning	Access?
No purchase	Student has not paid or unlocked the test	Only if price is zero
`pending`	Order created, payment not confirmed	No
`paid`	Razorpay signature verified	Yes
`simulated`	Manual UPI reference accepted or admin-confirmed	Yes
`manual`	Admin unlocked the test without payment	Yes

`Q266` Why is the amount multiplied by 100?

Razorpay expects the smallest currency unit, paise for INR. A ₹49 test is sent as 4900. The stored Purchase amount remains the human-readable rupee value.

`Q267` What prevents a forged Razorpay success?

The server recomputes the HMAC-SHA256 signature using the secret key and the exact `orderId|paymentId` string. A forged browser response cannot produce the correct signature without the secret. The server also checks the order belongs to the expected test and user flow.

`Q268` What is the weak point of the simulated payment?

A student can enter any non-empty UTR and receive access immediately. The server cannot validate the UPI transfer, so the flow relies on trust and later admin reconciliation. For a real business, a payment gateway webhook or manual approval before unlocking would be safer.

5.4

Certificate Eligibility

Q269 What exactly counts as course completion?

For the requested course: there must be at least one note and at least one test; every note must be in `readNotes` ; and every matching test must have a Result for that user. If any condition fails, progress returns incomplete with the missing counts.

Q270 How does the system decide which tests belong to a course?

It uses a case-insensitive regular expression on the test's `subject` . BCA matches "BCA"; BBA matches "BBA"; Biotech matches "BIOTECH", "BIOTECHNOLOGY" or the standalone word "BIO". This is why subject naming discipline matters: a typo can exclude a test from the certificate requirement.

Q271 When is the certificate actually created?

On the progress request, not on a separate "issue" button. The first time a student completes the course and asks for progress, the backend creates the Certificate with its unique ID. Later requests return the existing record.

Q272 Why is certificate creation not fully race-safe?

The check-then-create sequence is not atomic and there is no compound unique index on `userId + course` . Two simultaneous requests could both see "no certificate" and insert. The unique `certificateId` prevents identical IDs, not duplicate course certificates. A compound unique index or an upsert would fix it.

5.5

Note Ingestion Pipeline

Q273 Trace a PDF upload to a readable note.

1) Admin selects a PDF and metadata. 2) Multer receives it in memory. 3) The backend opens an upload stream to the `pdfs` GridFS bucket and writes the buffer. 4) `pdf-parse` extracts text from the same buffer. 5) A Note document is created with the file ID, text, course, subject and `hasContent`. 6) The student list shows the note; the viewer renders the text and links the original PDF.

Q274 What happens if text extraction fails?

The note can still be saved as a PDF-only record with empty content, so the viewer shows a message and offers the PDF download. A scanned PDF with no text layer will behave this way. OCR would be needed to make it searchable and readable in the HTML view.

Q275 How is a note deleted?

The route deletes the GridFS file by `fileId` and then deletes the Note document. This keeps the bucket from accumulating orphaned files. If the document delete fails after the file delete, the record can point to a missing file, so a transaction or ordered cleanup with logging is preferable.

5.6

AI Request Flow

Q276 Trace one chatbot message.

1) The student types a message and the component adds it to local state. 2) It POSTs `{ message, language }` to the AI chat endpoint. 3) The backend builds a system prompt plus a language instruction and calls Groq with the model and a temperature setting. 4) It extracts the assistant text and returns it. 5) The component adds the bot bubble and optionally speaks it.

Q277 What is sent to Groq?

A system message describing the assistant's role, the current user message and a language instruction. Conversation history is not sent, so the model has no memory of earlier turns. The API key is read from `GROQ_API_KEY` on the server and never exposed to the browser.

Q278 How would you add multi-turn memory?

Keep a messages array in React, send the last N turns to the backend, and have the backend forward them as the OpenAI-style messages list. For persistence, store a conversation ID and messages in MongoDB, with a limit on history length and token usage.

5.7

Admin Content Workflow

Q279 **What is the normal content workflow?**

Admin logs in → dashboard overview → upload notes or PYQs → create a mock test from a PDF or manually → confirm purchases if needed → students see the new content immediately because all lists are read live from the API. There is no draft/publish workflow or scheduled release.

Q280 **Why does the dashboard need a refresh after uploads?**

Each section loads its own list when it mounts or when an action succeeds. There is no global cache or real-time subscription, so a newly uploaded item appears when the relevant section refetches. This is simple and predictable, but a shared data store would avoid duplicate loading logic.

PART 06

06

Security, Limitations and Improvements

What is done well, what should be fixed before launch, and a prioritised improvement roadmap.

6.1

Security Strengths

- Passwords are hashed with bcrypt and the hash is excluded from normal queries.
- JWTs are signed server-side and verified on every protected request.
- Mock-test answers are hidden from the list API and scoring is recomputed on the server.
- Result and certificate reads check ownership, not just authentication.
- OTP codes are hashed, expire in 10 minutes, have a resend cooldown and an attempt limit.
- Password-reset tokens are short-lived and tied to a stored token ID, making them one-time.
- Razorpay payments are verified with an HMAC signature.
- Note text is HTML-escaped before rendering.
- HTML code preview runs inside a sandboxed iframe.
- The Groq and payment secrets stay on the server.

6.2

Security Gaps and Bugs

Issue	Impact and fix
Hard-coded admin credentials	The seed script creates a known admin account. Remove the literal credentials, require a strong password from an environment variable, and rotate the existing password. Never include the credential in documentation.
Inconsistent admin authorisation	`/api/admin/stats`, `/api/admin/students`, `/api/admin/courses` are public; notes/PYQ upload/delete and mock-test upload/add check only for a token. Add a reusable `isAdmin` middleware to every admin and content-write route.
Open CORS	Any origin can call the API. Restrict to the deployed frontend domain.
Token in localStorage	XSS can steal the token. Prefer an HTTP-only, Secure, SameSite cookie with CSRF protection, or at minimum a strict CSP and short-lived tokens.
No rate limiting	Login, signup, OTP, AI and file uploads can be abused. Add per-IP and per-user limits, especially for AI and OTP.
Signup password policy	The strong-password function is enforced on reset but not on registration. Apply it in both places.
No email verification	Anyone can register with an address they do not own. Add a verification link or OTP before full access.
Upload validation is minimal	Multer has no MIME-type filter or file-size limit. A user can upload large or unexpected files. Add a PDF-only filter, a size cap and a safe filename.
Memory uploads	A very large PDF is held in RAM before GridFS. Set a limit to avoid memory exhaustion.
No global error handler	Unexpected errors can leak stack traces or return HTML. Add central JSON error handling and avoid exposing internals.
Public AI endpoints	The Groq quota can be consumed by anyone. Require a token, limit message length and add rate limiting.
Prompt injection	User text can try to override the system prompt. Treat the system prompt as behaviour guidance, not a security boundary, and filter sensitive requests.
Hard-coded localhost URLs	The chatbot breaks after deployment. Use relative `/api` paths or an environment variable.
Certificate race	Two simultaneous progress calls can create duplicates. Add a compound unique index on user + course and use an upsert.
Manual UTR trust	A fake reference can unlock a test. Require admin approval or a gateway webhook.
No audit log	Admin unlocks and content deletions are not attributable. Record actor, action, target and timestamp.
No pagination	Student, purchase and content lists grow without bound. Add page/limit and indexes.
No attempt limit / autosave	Students can submit repeatedly and lose answers on refresh. Define attempt rules and persist progress.

> `Q281` **If you had one day before launch, what would you fix first?**
>
> Remove the hard-coded admin credential, add `isAdmin` to every admin/content-write route, restrict CORS, add upload limits and rate limiting, fix the chatbot's localhost URLs, and apply the strong-password check to signup. These are high-impact, low-effort fixes.

6.3

Code Quality and Maintainability

- `AdminDashboard.jsx` is very large; split it into one component per section.
- `components/Sidebar.jsx` and `AdminCard.jsx` are empty files; remove them or implement them.
- `AIExplainModal.jsx` is unused, and `BCARoadmap.jsx` duplicates `roadmapbca.jsx`; delete dead code.
- Repeated Axios/fetch logic and token handling should move into a shared API client with interceptors.
- Branding is inconsistent: "Mentor Lab" in the UI, "Drona Gyan" in the footer and `drona-gyan` in `package.json`.
- Some UI copy mixes English and Hinglish; choose one tone or localise properly.
- Date/number formatting and score parsing are duplicated across analytics, profile and certificates; extract helpers.
- Add unit tests for scoring, certificate eligibility, password reset and payment verification; add integration tests for the main API flows.

6.4

Prioritised Roadmap

Priority	Work
P0	Remove seeded credentials; add admin middleware everywhere; fix localhost AI URLs; add upload limits and rate limits; restrict CORS.
P1	HTTP-only cookies, email verification, signup password policy, pagination, API error envelope, compound certificate index, server-side exam autosave.
P2	Dedicated analytics endpoint, lecture CMS, backend code runner, certificate verification page/QR, admin audit log, real-time dashboard.
P3	Search and filters, accessibility audit, automated tests, CI/CD, observability, backup/restore plan.

PART 07

07

Rapid-Fire Answers and Demo Script

Last-minute revision, thirty one-line answers, likely viva traps and a ten-minute live demonstration plan.

7.1

Thirty Rapid-Fire Questions

Question	Answer
What is the project?	A MERN education platform for BCA, BBA and Biotech students with notes, PYQs, mock tests, certificates, analytics and AI help.
What does MERN stand for?	MongoDB, Express, React, Node.js.
Which database is used?	MongoDB through Mongoose.
Why MongoDB?	Flexible documents suit nested test questions, activity arrays and varied content metadata.
How is the frontend built?	React 19 with Vite, React Router and plain/CSS modules plus some component-level styles.
How does the frontend talk to the backend?	Axios/fetch to `/api`, proxied by Vite in development.
How are passwords stored?	bcrypt hashes with 10 salt rounds; never plain text.
What is JWT used for?	Stateless authentication; the token carries the user ID and expires in 7 days.
Where is the token stored?	`localStorage`, sent as a Bearer header.
How is admin access checked?	Client-side `AdminRoute` plus `req.user.isAdmin` on some server routes; coverage is inconsistent and must be fixed.
How does signup work?	Validate, check duplicate email, hash password, create user, return JWT.
How does password reset work?	Email OTP → verify → short-lived reset JWT → set new password.
How long is the OTP valid?	10 minutes, with a 60-second resend cooldown and limited attempts.
How are notes stored?	PDF in GridFS plus extracted text in the Note document.
How is note text rendered?	`renderPlainText` escapes HTML and converts plain text to paragraphs/headings.
How are PYQs organised?	By university, then course and subject; PDFs come from the `pyqs` GridFS bucket.
How is mock-test access controlled?	Free or purchased; `hasAccess` checks price and the Purchase collection.
How is a paid test unlocked?	Razorpay signature verification or manual UPI UTR/admin unlock.
How is the mock test scored?	+4 correct, -1 wrong, 0 unattempted, calculated on the server.
Why server-side scoring?	Prevents answer-key leakage and client-side score tampering.
How is the exam timed?	A one-second interval decrements the remaining seconds and auto-submits at zero.
What is the question palette?	Green answered, red unanswered, yellow marked for review.
How is a certificate earned?	Read every note and attempt every test in the course.
What is the certificate ID format?	`ML-YYYYMMDD-8 hex characters`.
How is the certificate downloaded?	Browser print dialog → Save as PDF.
Which AI provider is used?	Groq's OpenAI-compatible API with `openai/gpt-oss-20b`.
How does Hindi TTS work?	Text is chunked and proxied through Google Translate TTS; English uses browser speech synthesis.
Which compiler service is used?	Wandbox, called directly from the browser.

What is GridFS?	MongoDB's specification for storing large files as chunks plus metadata.
What would you improve first?	Admin authorisation, seeded credentials, rate limiting, upload validation and deployment URLs.

7.2

Ten-Minute Demonstration Script

1. **Pitch (30s):** "Mentor Lab is a MERN learning platform for BCA, BBA and Biotech students. It combines structured notes, previous-year papers, paid and free mock tests, progress analytics, certificates and an AI study assistant."
2. **Home and navigation (45s):** Show the hero, courses, QR signup, navbar and the three course areas. Mention that public pages need no login.
3. **Student signup/login (45s):** Register or log in, show the token/user in local storage, and explain JWT and bcrypt in one sentence each.
4. **Notes (60s):** Open Study Notes → BCA → a category → a note. Show the TOC, reading progress and original PDF link. Explain GridFS and text extraction.
5. **PYQ (45s):** Choose a university, show the course columns, and open a paper. Explain the GridFS `pyqs` bucket.
6. **Mock test (2 min):** Start a free test, answer a few questions, mark one for review, show the palette and timer, submit, and show the scorecard. Explain +4/-1 and server-side scoring.
7. **Paid test (45s):** Open a paid test, show the payment modal, explain Razorpay HMAC verification and the UPI fallback. Do not perform a real payment.
8. **Review and analytics (45s):** Open Test Review, then Analytics. Explain accuracy, trend and subject grouping from activity data.
9. **Certificate (45s):** Show the eligibility rule and a certificate view; use Print → Save as PDF.
10. **AI and compiler (45s):** Ask the chatbot a course question, switch to Hindi, and run a small Python/C program in the compiler.
11. **Admin (60s):** Log in as admin, show stats, upload a note or test, and show the student/purchase sections. Mention that some admin endpoints still need hardening.
12. **Close (30s):** Summarise the architecture, the strongest features and the P0 improvements. End with one measurable future goal.

7.3

Likely Viva Traps

Q282 "If the score is calculated in React, can I change it?"

No. The result shown is what the server returned. The submit endpoint recomputes from the database test, and the list endpoint omits `correctAnswer`. The frontend only renders the response.

Q283 "Why not store the JWT in a cookie?"

A cookie can be HTTP-only, so JavaScript cannot read it, which reduces XSS token theft. It needs CSRF protection and correct SameSite/Secure settings. The current localStorage approach is simpler but weaker.

Q284 "Is the AI generating the quiz questions?"

No. Tests are created by the admin manually or by parsing an uploaded PDF. The AI is only used for chat, explanations and text-to-speech.

Q285 "How do you know a student completed the course?"

The backend compares the course's note count with the student's `readNotes`, and the course's test count with the student's Result records. Both must match and be non-zero.

Q286 "What happens if two students have the same name?"

Email is the unique login identifier. Names are not unique; student ID is stored for display but is not used as the authentication key.

Q287 "Can the app run without internet?"

The main MERN app can run locally, but the AI chatbot, Hindi TTS and code compiler need internet because they call Groq, Google and Wandbox.

Q288 "What is the biggest limitation of the project?"

Authorisation consistency and production hardening. The feature set is broad, but admin APIs, uploads, rate limits and deployment URLs need tightening before a public launch.

Q289 "What did you learn?"

How to connect a React SPA to an Express/MongoDB API, model paid access and scoring securely, handle file storage with GridFS, and reason about authentication, ownership and third-party integrations.

Final one-line pitch: Mentor Lab is a full-stack learning platform that brings curriculum notes, previous-year papers, practice tests, payments, certificates, analytics and AI assistance into one student workspace.